Efficient Computation of Optimal Thresholds in Cloud Auto-scaling Systems

THOMAS TOURNAIRE and HIND CASTEL-TALEB, SAMOVAR, Télécom SudParis, Institut Polytechnique de Paris, 91120 Palaiseau, France
EMMANUEL HYON, UPL, Université Paris Nanterre, Nanterre, France and LIP6, Sorbonne Université, CNRS, Paris, France

We consider a horizontal and dynamic auto-scaling technique in a cloud system where virtual machines hosted on a physical node are turned on and off to minimise energy consumption while meeting performance requirements. Finding cloud management policies that adapt the system to the load is not straightforward, and we consider here that virtual machines are turned on and off depending on queue load thresholds. We want to compute the optimal threshold values that minimize consumption costs and penalty costs (when performance requirements are not met). To solve this problem, we propose several optimisation methods, based on two different mathematical approaches. The first one is based on queueing theory and uses local search heuristics coupled with the stationary distributions of Markov chains. The second approach tackles the problem using Markov Decision Process (MDP) in which we assume that the policy is of a special multi-threshold type called hysteresis. We improve the heuristics of the former approach with the aggregation of Markov chains and queues approximation techniques. We assess the benefit of threshold-aware algorithms for solving MDPs. Then we carry out theoretical analyzes of the two approaches. We also compare them numerically and we show that all of the presented MDP algorithms strongly outperform the local search heuristics. Finally, we propose a cost model for a real scenario of a cloud system to apply our optimisation algorithms and to show their practical relevance. The major scientific contribution of the article is a set of fast (almost in real time) load-based threshold computation methods that can be used by a cloud provider to optimize its financial costs.

CCS Concepts: • **Computing methodologies** → *Modeling and simulation*; • **Applied computing** → *Operations research;*

Additional Key Words and Phrases: Decision processes, Markov processes, hysteresis queues, heuristics, cloud system

ACM Reference format:
Thomas Tournaire, Hind Castel-Taleb, and Emmanuel Hyon. 2023. Efficient Computation of Optimal Thresholds in Cloud Auto-scaling Systems. *ACM Trans. Model. Perform. Eval. Comput. Syst.* 8, 4, Article 9 (July 2023), 31 pages.
https://doi.org/10.1145/3603532

Authors' addresses: T. Tournaire and H. Castel-Taleb, SAMOVAR, Télécom SudParis, Institut Polytechnique de Paris, 91120 Palaiseau, France; emails: tournaire.thomas@hotmail.fr, hind.castel@telecom-sudparis.eu; E. Hyon, UPL, Université Paris Nanterre, Nanterre, 92000, Nanterre, France and LIP6, Sorbonne Université, CNRS, Paris, 75005 Paris, France; email: emmanuel.hyon@parisnanterre.fr.
Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
© 2023 Copyright held by the owner/author(s). Publication rights licensed to ACM.
2376-3639/2023/07-ART9 $15.00
https://doi.org/10.1145/3603532

1 INTRODUCTION

The rapid growth in the demand for computational power by scientific, business, and web applications has led to the creation of large-scale data centers that are often oversized to ensure the quality of service of hosted applications. This leads to an under-utilisation of the servers that causes a significant over-consumption of electricity. Nowadays, cloud computing requires more electric power than that required by whole countries such as India or Germany [30, 32].

To improve the utilisation rate of servers, some data-center owners have deployed methods implementing dynamic resource allocation according to system load, called "*autoscaling*." More specifically, horizontal autoscaling [28] is based on activation and deactivation of **Virtual Machines (VMs)** [7, 10] according to the workload. Moreover, cloud providers such as AWS, Azure, and others use horizontal autoscaling with thresholds based on metrics (such as CPU utilization or the number of request) to trigger the increase or decrease in the number of VMs. However, due to workload variations, manually setting thresholds by users is not effective in ensuring both a level of **Quality of Service (QoS)** for applications and a good use of resources. That is why cloud providers have proposed dynamic threshold adjustment strategies depending on load variation [27].

To dynamically adapt resources, cloud systems can modulate the service capacity according to multiple queue occupancy thresholds (for scaling up and scaling down). More precisely, activations and deactivations of servers are ruled by sequences of different forward and reverse thresholds: These are hysteresis models [5, 8, 22]. When the load is varying, identical activation and deactivation thresholds cause an oscillation phenomenon of the system that is avoided with hysteresis models. The other advantage that makes hysteresis policies so appealing is their ease of implementation in cloud systems. This is why they are a key component of the auto-scaling systems for the widespread cloud architectures Kubernetes for docker components and Azure or Amazon Web Services [1] for virtual machines. Hence, there is a great interest in studying the computation of hysteresis policies, since threshold values can be plugged into autoscaling systems that are implemented in the major cloud architectures.

Unfortunately, finding the right values of the thresholds is not easy [28] and requires some cloud system analysis and some optimisation studies. Hence, it is crucial to analyse both the energy consumption and the performance of the system [14] to find the policy that adapts resource allocation to the demand. We point out that these two measures are inversely proportional, which motivates researchers to evaluate them simultaneously via a unique global cost function. Therefore, we consider a cost-aware approach and propose a global expected cost function that takes into account different costs: costs associated with the performance requirements defined in a **Service Level Agreement (SLA)** and costs associated with energy consumption. Including energy saving aspects in the global cost function is relatively new. Indeed, the review of auto-scaling techniques [28] does not mention such criteria. However, the performance analysis of cloud systems is performed with the use of a multi-server queueing system working with thresholds-based policies representation. These models have been suggested [5, 29, 40], since they can represent cloud system with a high precision and since they efficiently manage the number of active VMs.

In this work, we only focus on the queue occupancy thresholds that trigger the increase or decrease in the number of active VMs. Cloud Management, placement of VMs on physical machines, and migration of VMs are considered out of scope. We want to compute the optimal policy of activation and deactivation of the servers so as to minimize energy consumption while meeting SLA. Thresholds depend on the queue occupancy, which is the number of customers in a multi-servers queue. Assuming the policy has a hysteresis property (i.e., forward and reverse threshold sequences are different) is not sufficient to fully characterize the optimal policy, and we have to determine the concrete values of thresholds. One of our objectives is to allow the cloud

provider to calculate very quickly and accurately the dynamic thresholds in the planning phase from prediction models for various values of the load. This is why a special concern is brought to the speed and efficiency of the algorithms. Running time evaluations were carried out to estimate the real-time applicability of the methods.

To compute the exact optimal values of thresholds in a multi-server queueing system, two main optimization approaches are possible, as stated in the literature. The first one expresses the problem as a *deterministic optimisation problem*. This can be performed by using the average value of the costs computed by stationary distribution of a **Continuous Time Markov Chains (CTMC)**. Such an approach is often used in the field of inventory management for base stock policies [42] or reliable servers [43]. The second one expresses the problem as a **Markov Decision Process (MDP)** model with structured policies and computes the optimal one.

For the first approach, some previous works (Reference [31] (for a reserve of servers) or Reference [35]) studied the computation of the stationary distribution. Reference [12] presents threshold activation and deactivation methods that are very similar to ours and that we will use as baseline. Furthermore, Reference [40] makes an extensive study of the heuristics that solve the deterministic optimisation problem. We propose here an improvement of the best heuristics identified in Reference [40]. For the second approach using MDP, most of the works mainly focused on showing the optimality of hysteresis policy for cloud models [46] but, to our knowledge, none has focused on the threshold computation and the running time efficiency. This is done here where we want to determine the best algorithm. To this aim, we assess the running time efficiency of the algorithms of each of the two approaches considered here. Although they are widely used for threshold computation, their effectiveness has not been carefully evaluated. This was done for single thresholds by level policies in Reference [36], but this had never been done for hysteresis policies. Determining which is the best modelling approach (**Markov chain (MC)** or MDP) and the most efficient and accurate technique to compute the thresholds minimizing the global cost function is thus an important point that is addressed here.

The contributions of this article are as follows:

(1) We improve the existing heuristics of Reference [40] for the deterministic model by modifying several steps of the previous heuristics leading to new efficient algorithms. We assess numerically the gain given by these improvements;
(2) We provide a theoretical study of the multichain property of the Markov Decision Process and we propose an algorithm that solves it with a convergence guarantee. We also design MDP algorithms assuming the policy is of hysteresis type, and we show that they are at least twice faster than traditional dynamic programming algorithms while ensuring the optimality of the solution;
(3) We perform a large set of numerical experiments which demonstrate that the MDPs strongly outperforms heuristics using the Markov chain. The comparison criteria are the precision and the running time. We perform a theoretical analysis of the two models and give some insights explaining why using deterministic optimisation approach is suboptimal;
(4) We develop and analyse a financial cost model. It takes into account prices of VM instantiations from cloud providers as well as energy consumption of VMs. For a cloud model based on these concrete values, we present the behaviour of the optimised global costs. We noticed that optimal thresholds are generated in just few seconds, even for large systems that could have a significant impact for a cloud operator.

The remainder of this article is organised as follows. Section 2 briefly reviews the related works. In Section 3, we provide a detailed presentation to unify the different definitions of hysteresis policies coming from different works. Then we describe a cloud system, represented

by a multiserver queueing model. We detail the cost function used to express the expected costs in terms of performance and energy consumption for such models. In Section 4, we present the Markov chain approach with the decomposition and aggregation method and the optimisation algorithms based on local search heuristics. We also present a Markov Decision Process model and adapted algorithms in Section 5. In Section 6, we theoretically compare the two approaches. Then Section 7 presents a concrete cost model. Numerical experiments for the comparison of the algorithms are discussed in Section 8. In the conclusion, we finally discuss achieved results. Some comments about further researches are given.

1.1 Discussion

The main objective of this article is to propose solutions for the fast computation of auto-scaling thresholds in cloud systems. We develop and compare two different mathematical approaches to compute the thresholds minimizing the cost function. The goal of the two approaches is to minimise operational costs for cloud providers regarding energy consumption and performance costs. Although similar techniques have been applied for models with single level thresholds in the literature, we propose, in this work, to extend to a model with multiple thresholds as well as to analyse and compare methods that compute the values of the thresholds. The first approach based on Markov chains and heuristics has been first studied in Reference [40], yet many improvements have been added for the heuristics and are presented in this study. Combining heuristics based on local searches with suitable initialization of the thresholds, and aggregation of the Markov chain, allow us to reduce the computational time and improve the accuracy of the algorithms. We believe that an other approach can be more efficient in terms of execution time and accuracy, which is crucial for cloud providers to dynamically recompute the whole thresholds when the queue parameters change. This is why, in this work, we compare the first approach to the second one, which uses Markov Decision Process. Since the two formalisms are different regarding the optimisation problem, we need precise comparison, both theoretical and numerical, to understand which method is the more efficient one for cloud providers.

We summarise now the key findings of this study. The first conclusion is that the Markov decision process approach is more efficient than the Markov chain plus heuristics one. Indeed, the theoretical analysis points out major differences between the two approaches. The MDP formalism does not have specific constraints on the structure of the hysteresis policy, whereas the MC approach assumes constraints on the hysteresis policy and solves an optimisation problem under constraints. This implies that the solutions found by MDP algorithms are more general, and such techniques can solve more cloud systems with different queuing parameters. Second, the numerical experiments strengthen the theoretical analysis and show that MDPs have better performances for both execution time and accuracy. Finally, to understand the importance of the MDP approach, we present in this article the execution time of these algorithms in large-scale cloud system to show that such approach can be used dynamically in real systems and can quickly adapt to changes in the system.

2 LITERATURE REVIEW

2.1 Dynamic Energy and Performance Management in the Cloud

In recent years, the increase in the energy consumption has remained a crucial problem for large-scale computing. In a data center, the server power consumption can be divided into static and dynamic parts. The static part (which does not vary with workload) represents the energy consumed by a server when it is idle, while the dynamic cost depends on the current usage. A power-aware model is defined in Reference [25] to estimate the dynamic part of energy cost for a VM of a given size. This model, which is based on energy consumption, keeps the philosophy of the pay as you

go model. Since the static part represents a high part of the overall energy consumed by the server nodes, shutting unused physical resources that are idle lead to non-negligible energy savings. Different and complementary physical server resource management approaches have been proposed for dynamic power management to improve the energy efficiency. These include shutdown and switching on servers (on/off) [7] and resource auto-scaling. Hence this subject is therefore very topical today.

2.1.1 Auto-scaling Strategies. Auto-scaling is a technique that automatically adjusts resources provisioned to applications according to real-time workloads without human intervention. Its impact is crucial, because it allows application providers to minimize the financial cost of using cloud resources while guaranteeing the user QoS constraints. There are different kinds of scaling policies. Horizontal scaling, which is widely used by cloud service providers, corresponds to launching or releasing new VMs. Vertical scaling means removing or adding resources to existing VMs. Finally, there is Dynamic Voltage and Frequency Scaling [24]. Auto-scaling of VMs can be combined with VM migrations to consolidate the load on fewer nodes and then shut off physical servers that become idle to save energy [7, 21].

Reference [28] presents a large review on the auto-scaling policies and the techniques to determine the right amount of resource to be switched off or on. These techniques are grouped into several frameworks: rules, queueing theory, reinforcement learning, automatic control, and time series. Threshold-based techniques belong to several of them (rules, queueing theory, and control). They are very popular and easy to implement for scaling in/out the resources according to performance measures (number of requests, CPU load,...). In general, infrastructure as a service providers allow users to set the threshold values for triggering the automatic increase or decrease of virtual machine resources. Unfortunately, fixed thresholds for triggering virtual machine resources is not an optimal approach, because the workload varies with the time and the kind of applications in use. In Reference [27], they propose a dynamic threshold adjustment strategy and they integrate their technique on the AWS platform. For the experimentation, they implement a web application on the AWS platform, and they used Apache JMeter to compare the response times of the fixed threshold.

Implementing an auto-scaler in cloud data centers is challenging [33], as the auto-scaler repeats continuously over time a control cycle containing the following phases: monitoring some performance indicators, analysing scaling actions, planning of the resource, and executing the scaling. Each phase is quite complex due to the size of the data center, the diversity of applications, and their constraints. The monitoring could be performed at different levels: application and infrastructure. A multi-level monitoring framework to meet the whole spectrum of monitoring requirements for containerized self-adaptive applications is proposed [38]. They propose a fine-grained auto-scaling method with dynamic thresholds to improve resource management. In the analysis phase, queueing theory is very efficient to estimate the performance metrics that are used during the planning phase namely to compute the optimal thresholds (guaranteeing QoS and controlling the resource use).

To represent the problems with activations and deactivations of virtual machines, server farm models have been proposed [9, 31]. Usually, these server farm models are modeled with multi-server queueing systems [3, 4]. Although server farms with multi-server queues allow a fairly fine representation of the dynamicity induced by virtualization, these models do not address issues related either to the internal network nor to the VM placement in it. All VMs are instead considered as parallel resources. This makes these models suitable for studying simple nodes of several servers in the cloud. If queuing models allow us to easily compute performance metrics, then the decision making for switching VMs on or off requires an additional step, which remains a key point.

2.2 Markov Decision Process and Autoscaling

Dynamic control and especially Markov Decision Processes are used in a lot of works about autoscaling and cloud management. Hence in References [44, 45], finite Horizon MDP have been used to solve the problem of deactivating optional micro-services for saving energy and promoting renewal energy. They show that MDP gives long term better results than heuristic-based algorithms. However, in these works only a similar algorithm than value iteration has been considered. Rather, the authors prefer to work on a roughly approximated model of the cloud to limit the size of the problem. If in Reference [45] we can estimate the size of the state space to 80,000 elements, then there is no information about the running times of algorithms. In Reference [13], an MDP is used for VM consolidation but, unlike us, the decision is related to the placement of the VM on the servers rather than the computation of the right number of VM that should be turned on. Also, only value iteration is used. Nevertheless, Reference [13] uses the framework of approximate dynamic programming to approach the optimal policy. Approximate Dynamic Programming is an interesting way that is out of scope of our work, since we focus on the improvement of exact methods. For computing the deactivation and activation in the cloud, we can refer to Reference [46], but this work aims to show the optimality of hysteresis policies.

In the field of multi-server queue models, the computation of the optimal policies with activation and deactivation of servers has been studied for a long time. Numerous works have been devoted to the use of Markov decision processes (e.g., see Reference [39] and references therein for the oldest and more recently References [46] and [26]). Unfortunately, not all of them received a rigorous study of unichain or multichain properties. Furthermore, no work focuses on improving the resolution algorithms. This is why the standard value iteration algorithm can be considered as the baseline of such algorithms, and this is why there is an interest to find some algorithmic improvements.

Research on hysteresis policies in which activations and deactivations of servers are ruled by sequences of different forward and reverse thresholds appear very early (see Reference [39]). However, research on hysteresis policies in MDP focused on exploration of the conditions that insure the optimality of hysteresis policy. Szarkowicz et al. [37] showed the optimality of hysteresis policies in a $M/M/S$ queueing model with a control of the vacations of the servers. For models with control of the service rates, the proofs are made in Hipp [15] or Serfozo [22].

Reinforcement learning is a modification of MDP models in which the statistics of the MDP are not known and should be learned. Reinforcement learning techniques based on Q-Learning is a model free approach that automatically and dynamically adjust the thresholds meeting QoS constraints. However, this method suffers from slow convergence due to state and action space explosion. To solve this problem, in Reference [16] they propose Q-Threshold a novel algorithm that exploits the monotonicity of the Q-Function to reduce action space, making the convergence faster. But this algorithm seems not to be so different from our algorithm called *double thresholds* while also being slower due to learning aspects.

2.3 Threshold Calculation Methods in Queueing Systems

The calculation of optimal thresholds in queueing systems received less attention in the past, but two major models appeared. The computation of the optimal policy can be performed by means of adapted usual dynamic programming algorithms in which the structured policies properties are plugged. To our knowledge, no work clearly assesses the advantage of this approach comparatively with value iteration. The alternate way is similar to the research for the set of single thresholds in base-stock policies (see, e.g., Reference [23]). The problem is expressed as a deterministic optimization problem. A search is required to explore the space of thresholds, and for each set

of thresholds the computation of expected measures is performed. This requires the computation of the invariant distribution and the expected cost for numerous threshold values that is very time consuming. Moreover, the computation of the stationary distribution is complex. It can be performed either by standard numerical methods, see, e.g., Reference [43], or after a Markovian analysis with simpler and faster computations (e.g., Reference [42] uses iterative methods). For the optimisation problem, since the objective cost function is non-linear and non-convex, this problem is very complex, and there is currently no exact method to solve it. Reference [23] presents three heuristics for searching the set of single thresholds in inventory models. These heuristics have been adapted for hysteresis in Reference [40]. For a server farm model with activation of a single reserve threshold, Mitrani [31] uses fluid approximation to compute the only activation threshold and Reference [43] uses a genetic algorithm for optimisation and a matrix geometric method for the stationary distribution computation. Last, from Reference [28], we keep Reference [12], since it proposes a queueing approach to determine threshold values. This technique seems very similar to the *fluid* approach in Reference [31]. Here we will combine local search in Reference [40] and queueing analysis to improve the running time.

Song [36] claims that, for single-threshold models, the approach dealing with stationary distribution computations is generally more effective than the MDP approach. Such a comparison has not been performed yet for hysteresis especially since few works implement a MDP algorithm with a structured policy. This comparison is further complicated, because the mathematical definitions of the hysteresis properties are not similar in the two approaches.

Aggregation in Markov Chain. As cloud systems are modelled by multi-dimensional systems, defined on very large state spaces, the stationary distribution computation is difficult. Fortunately, the computation of the performance measures of hysteresis multi-server systems has been already studied. Different efficient resolution methods have been developed. Among the most noticeable works, we quote the work of Ibe and Keilson [17] refined in Lui and Golubchik [29] or in Reference [6]. Both solve the model by partitioning the state space in disjoint sets to aggregate the Markov chain. Exhaustive comparisons of the resolution methods are made in Reference [20]: Closed-form solution of the stationary distribution, partition of the state space, and matrix geometric methods applied on Quasi Birth and Death processes are studied. It was noticed that partitioning is the most suited technique.

3 CLOUD MODEL

In this section, we propose to study a cloud system representing a physical server hosting VMs where the auto-scaler dynamically manages the allocation of the VMs according to thresholds. We define the *Cloud Model* (see Figure 1 and Table 1) as a system modelled by a continuous time-controlled multi-server queueing model where arrivals of requests are processed by servers. The servers in the queueing system represent logical units in the cloud. Since we consider Infrastructure as a Service that provide computing resources as a service, the logical units are either virtual machines or docker components. We propose a mathematical analysis of the queueing model to derive performance as well as energy consumption measures.

In the following, first we present in details the controlled multi-server queueing model and the global cost minimisation problem, and then we focus on the optimal control based on threshold hysteresis policies.

3.1 Controlled Multi-server Queue

The cloud model is a multi-server queueing system where each VM (or evenly a core each) represents a server. We have the following assumptions concerning the continuous time model:

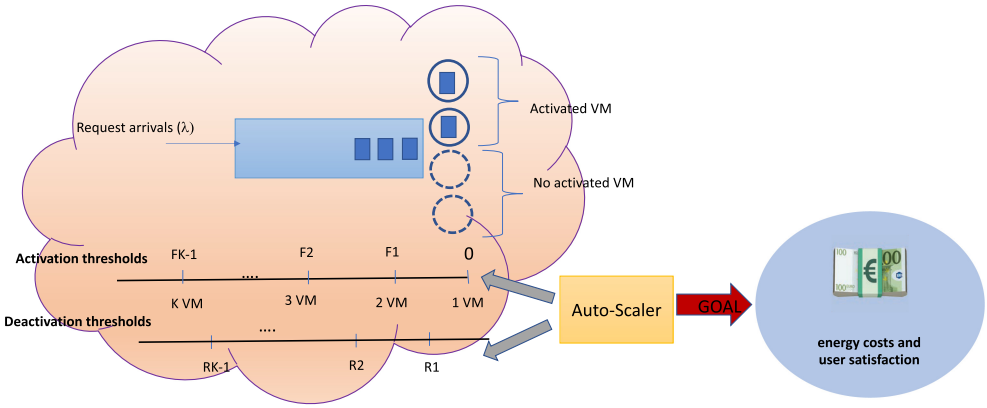


Fig. 1. The cloud system.

(1) Arrivals of requests follow a Poisson process of rate λ, and service times of all VMs are independent of arrivals and independent of each other. Moreover, they are i.i.d. and follow an exponential distribution with an identical rate μ;
(2) Requests enter in the queue, which has a finite capacity B and are served by one of the K servers. The service discipline is chosen to be First In First Out as usually in cloud.

A customer is treated by a server as soon as this server becomes idle and the server is active. Servers can be turned on and off by the controller. When the server is turned on it becomes active while it becomes inactive when it is turned off.

We define $\mathcal{S}$ the state space, where $\mathcal{S} = \{0, 1, \ldots, B\} \times \{1, \ldots, K\}$. Any state $x \in \mathcal{S}$ is such that $x = (m, k)$, where m represents the number of requests in the system and k is the number of operational (or active) servers (this number can also be seen as a service level). We focus, in this work, on models in which we can only operate one machine at a time. Thus, the action indicates either an activation or a deactivation or no change. We define $\mathcal{A} = \{-1, 0, 1\}$ to be the set of actions such that for a state (m, k), when the action is -1 then we deactivate a server, when the action is 0 then the number of active servers remains identical, and when the action is 1 then we activate a server.

With this system are associated two kinds of costs that a cloud provider encounters:

(1) Costs corresponding to the performance of the system: These are for the control of the service quality defined in the SLA. We have costs per unit of time for holding requests in the queue (we count C_H per request and per unit of time) as well as instantaneous costs for losses of requests (we count C_R each time a loss occur).
(2) Costs corresponding to the use of resources (operational and energy consumption): We have costs for using a VM per unit of time (we count C_S per activated VM per unit of time) and instantaneous costs for activating/deactivating (we count C_A for an activation and C_D for deactivation).

We define the long term expected cost $\bar{C}$ as the expectation of all costs collected during the system works. This global cost, containing both performance and energy consumption costs, defines the objective function that we want to minimise. It is defined by the following:

$$\bar{C} = \lim_{T \to \infty} \frac{1}{T} \mathbb{E}\left\{ \sum_{t=0}^{T} C\left(x_t, q_t, \tau_t\right) \right\}, \tag{1}$$

where $C(x_t, q_t, \tau_t)$ is the charged cost at the tth transition time when the state of the system is x_t, the action triggered is q_t and the duration time time of the tth transition is τ_t. The charged cost

Table 1. Notation and Acronyms

Notation	Description
λ	request arrival rate
μ	request service rate
B	system capacity
K	number of servers (VMs)
$[F_1, F_2, \ldots, F_{K-1}]$	activation thresholds of the Markov chain
$[R_1, R_2, \ldots, R_{K-1}]$	deactivation thresholds of the Markov chain
$(L_2, \ldots, L_K)$	activation thresholds of the Markov Decision Process
$(l_2, \ldots, l_K)$	deactivation thresholds of the Markov Decision Process
$\mathcal{A}$	Set of actions
$\mathcal{S}$	$\{0, 1, \ldots, B\} \times \{1, \ldots, K\}$
$\mathcal{X}$	state space of the Markov chain
$x = (m, k)$	system state, m : number of requests, k number of operational servers
q	decision rule
q^*	decision rule returning the optimal policy
$\{X(t)\}_{t \geq 0}$	Markov chain
π	stationary probability distribution of the markov chain $\{X(t)\}_{t \geq 0}$
Π	stationary probability distribution of the *macro-chain*
π_k	stationary distribution of the *micro-chain* of level k
$\overline{C}^{\pi}_{[F,R]}$	global cost function for fixed thresholds $[F, R]$ and stationary probability distribution π
C_H	cost of holding a request in the queue
C_R	cost of rejecting a request
C_A	VM activation cost
C_D	VM deactivation cost
C_S	cost of using a VM
BPL	Best Per Level
NLS	Neighborhood Local Search
BPL-MMk	Best Per Level coupled with M/M/k/B approximation
BPL Agg	Best Per Level coupled with Markov chain aggregation
BPL-MMk Agg	Best Per Level coupled with M/M/k/B approximation and Markov chain aggregation
NLS-MMk	Neighborhood Local Search coupled with M/M/k/B approximation
NLS Agg	Neighborhood Local Search coupled Markov chain aggregation
NLS-MMk Agg	Neighborhood Local Search coupled with M/M/k/B approximation and Markov chain aggregation
Fluid Agg	Fluid approximation coupled with Markov chain aggregation
VI	Value Iteration
RVI	Relative Value Iteration
PI	Policy Iteration
PI Adapt	Policy Iteration modified Adapt
DL-PI	Double Level Policy Iteration
Hy-PI	Hysteresis Policy Iteration

$C(x, q, \tau)$ is a function of the different costs (C_H, C_R, C_S, C_A, C_D). If $x = (m, k)$, then the charged cost is equal to

$$C(x, q, \tau) = m \cdot \tau \cdot C_H + (k + q) \cdot \tau \cdot C_s + C_A \mathbb{1}_{\{q=1\}} + C_D \mathbb{1}_{\{q=-1\}}, \tag{2}$$

where $\mathbb{1}$ stands for the indicator function. If the transition is a rejection, then we should add C_R to the charged cost. The global cost represents the financial cost that the cloud provider must pay. The minimisation of this cost can be obtained with two different models. Each of them expresses the cost $\bar{C}$ in a different mathematical form that allows us to use different optimization techniques. When we use a Continuous Time Markov Chain to solve a deterministic optimization problem (see Section 4), we call this approach the MC approach or deterministic; when we use Semi Markov Decision Processes (see Section 5), we call this approach MDP.

3.2 Details on Hysteresis Policy Models

Throughout this work, we study a special form of activation and deactivation policies: the *hysteresis* policies. These hysteresis policies correspond with autoscaling policies with dynamic thresholds. We detail in this part the formal definition of such policies and the conditions of their optimality. We restrict our attention to the case of a single VM switching; similar definitions for multiple switchings are out of scope of the article and can be found in Reference [41].

As already stated, there exist two different mathematical definitions of such policies. It can refer either to policies defined with double thresholds [15, 22] in a controlled system or to policies defined with sequences of thresholds [29]. For a case of single VM switching, as considered here, we show below that the definition of Reference [29] is a special case of the formalism given in References [15, 22].

Hysteresis policies in a controlled model. In our controlled model, the action that should be performed in a decision epoch is given by a deterministic decision rule that maps the current state to an action. We thus consider the mapping q from $\mathcal{S}$ to $\mathcal{A}$. A policy defines the action to be performed at each decision epoch and is defined here as a sequence of the same single deterministic decision rule. Validity of such kinds of policies can be found in Reference [34].

We give now the definition of the double thresholds policy.

Definition 3.1 (Double thresholds policies [22]). We call a *double threshold* policy a stationary policy such that the decision rule $q(m, k)$ is increasing in m and decreasing in k and such that the decision rule is defined by a sequence of thresholds such that for any k in $[1, K]$ and $k_1 \in \{-1, 0, 1\}$ we have the following:

$$q(m, k) = k_1 \text{ when } \quad \ell_{k_1}(k) \leq m < \ell_{k_1+1}(k),$$

where $\ell_{k_1}(k) = \min\{m : q(m, k) \geq k_1\}$. This minimum is ∞ if the set is empty. For all k, we also fix $\ell_{K+1}(k) = \infty$ and $\ell_1(k) = 0$ (since at least one server must be active).

A monotone hysteresis policy is a special case of a double thresholds policy.

Definition 3.2 (Monotone hysteresis policy [15]). A policy is a monotone hysteresis policy if it is a stationary policy such that the decision rule $q(m, k)$ is increasing in m and decreasing in k and such that the decision rule is defined by two sequences of thresholds l_k and L_k such that for all $(m, k) \in \mathcal{S}$,

$$q(m, k) = \begin{cases} -1 & \text{if } m < l_k \text{ and } k > 1 \\ 0 & \text{if } l_k \leq m < L_{k+1} \\ 1 & \text{if } m \geq L_{k+1} \text{ and } k < m \end{cases},$$

with $l_k \leq L_k$ for $k = 1, \ldots, K + 1$; $l_k \leq L_{k+1}$ for $k = 1, \ldots, K$ and $l_1 = L_1 = 0$; $l_{K+1} = L_{K+1} = \infty$.

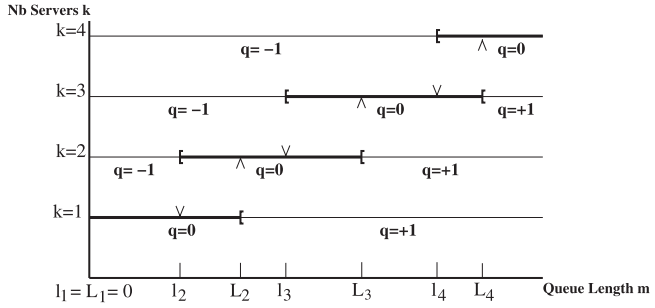


Fig. 2. An isotone hysteresis policy.

When the thresholds are ordered, we talk about isotone hysteresis.

Definition 3.3 (Isotone Hysteresis polices [15]). A policy is an isotone policy if it is a monotone hysteresis policy and if $0 = l_1 \leq l_2 \leq \cdots \leq l_{K+1} = \infty$ and $0 = L_1 \leq L_2 \leq \cdots \leq L_{K+1} = \infty$. When the inequalities are strict for all k, we say that the policy is *strictly isotone*.

Example 1. In Figure 2, we represent an isotone policy. The bold line means that the number of activated servers is optimal and no activation or deactivation should be done. The optimal decision q indicates the number of servers that should be activated or deactivated in each state. Hence, in level $k = 2$, if the number of customers m is smaller than l_2, then a server should be deactivated, and if m is larger than L_2, then a server should be activated.

Double level policies are policies in which the decision is identical between two consecutive thresholds. It could be noticed that in the case of single VM switching, double level policies and monotone hysteresis policies coincide. This is mainly due to the fact that there are only two thresholds by level k: L_{k+1} to go to level $k + 1$, and l_k to go to level $k - 1$. There are no other levels that can be reached from k. Instead, when one can activate or deactivate several servers at a time there are multiple levels that can be reached from a level k and therefore more than two thresholds in a given level k. In this case, the difference between a double threshold and a monotone hysteresis policy lies in the fact that thresholds toward a level are identical in hysteresis while they can be different in double level policies.

Optimality of hysteresis policy. Finding the optimal thresholds of a hysteresis yields to the optimal hysteresis policy. However, the optimal hysteresis policy cannot be the optimal activation and deactivation policy. Indeed, hysteresis policies do not cover the entire set of activation and deactivation policies. The following theorem insures that the optimal hysteresis policy is also the optimal activation and deactivation policy.

Proposition 1 (Optimality of Monotone Hysteresis Policies [37]). *In the multi-server queueing model presented in Section 3 and for which there is activation/deactivation costs, working costs and holding costs, Szarkowicz et al. showed that monotone hysteresis policies are optimal policies.*

However, the presence of a rejection cost, as we assume here, is not considered in the assumptions in Reference [37], and there is no proof about the optimality of hysteresis policies for the model studied here.

Hysteresis policies and Markov chain. As presented in Reference [39], there exists a different model of multi-server queue with hysteresis policy that received a lot of attention (see Ibe and Keilson [17], Lui and Golubchik [29], and Reference [6]). This model is still a multi-server queueing system but is no longer a controlled model: The transitions between levels are also governed by

sequences of prefixed thresholds, and this is why it is called a hysteresis model. It is built to be easily represented by a Markov chain. The differences between the controlled model and this Markov chain model are detailed in Section 6.

Remark. For a sake of clarity, deactivation and activation sequence of thresholds will be denoted respectively by R and F in the Markov chain model, while they will be denoted l and L in the MDP model.

Definition 3.4 (Hysteresis policy [29]). A K-server threshold-based queueing system with hysteresis is defined by a sequence $F = [F_1, F_2, \ldots, F_{K-1}]$ of activation thresholds and a sequence $R = [R_1, R_2, \ldots, R_{K-1}]$ of deactivation thresholds. For $1 \leq k < K$, the threshold F_k makes the system goes from level k to level $k+1$ when a customer arrives with k active servers and F_k customers in the system. Conversely, the threshold R_k makes the system goes from level $k+1$ to level k when a customer leaves with $k+1$ active servers and $R_k + 1$ customers in the system.

Furthermore, we assume that $F_1 < F_2 < \cdots < F_{K-1} \leq K$, $1 \leq R_1 < R_2 < \cdots < R_{K-1}$, and $R_k < F_k$, $\forall\, 1 \leq k \leq K-1$. We denote the vector that gathers the two threshold vectors F and R by $[F, R]$.

Remark. Note that with the above definition no server can remain idle all the time, since the threshold values are bounded. Instead, Definition 3.2 allows infinite thresholds for which the server remains idle. Hence a hysteresis policy presented in Reference [29] can be seen as a restricted version of an isotone policies given in Definition 3.2. Furthermore, the inequalities being strict in Reference [29], then the hysteresis of Definition 3.4 is a very specific case of hysteresis of Definition 3.2.

4 HYSTERESIS POLICIES AND MARKOV CHAIN APPROACH

This section is devoted to the study of the approach in which we transform the stochastic problem into a deterministic problem by mean of Markov chain. This approach is based on the policies defined in Definition 3.4. We detail the cloud system modeling, the expression of the optimisation problem, and the heuristics to solve it. We also present a computation of the invariant probability, based on some aggregation properties of the Markov chain, that reduces the execution time of the heuristics.

4.1 Hysteresis Queueing Model

In the hysteresis model of Definition 3.4, thresholds are fixed and do not change while the system works. The sequence of the valued thresholds is a characteristic of the model that is required to analyse the system's functioning. For a set $[F, R]$ of fixed thresholds, the stochastic model is represented by a CTMC denoted by $\{X(t)\}_{t \geq 0}$. Each state $(m,\, k)$ is such that m is the number of requests in the system and k is the number of active servers. The state space is denoted by $\mathcal{X}$ with $\mathcal{X} \subset \mathcal{S}$, since we do not consider non-reachable state of $\mathcal{S}$. Thus, the state space is given by the following:

$$\mathcal{X} = \{(m,\, k) \mid 0 \leq m \leq F_1,\ \text{if } k = 1\,, R_{k-1} + 1 \leq m \leq F_k, \text{if } 1 < k < K\,, R_{K-1} + 1 \leq m \leq B, \text{if } k = K\}. \tag{3}$$

The transitions between states are described by

$$\begin{aligned}
(m,k) &\to (\min\{B, m+1\}, k),\ \text{with rate } \lambda\,,\ \ \text{if } m < F_k\,;\\
&\to (\min\{B, m+1\}, \min\{K,\ k+1\}), \text{with rate } \lambda\,,\ \ \text{if } m = F_k\,;\\
&\to (\max\{0, m-1\}, k),\ \text{with rate } \mu \cdot \min\{m, k\},\ \text{if } m > R_{k-1} + 1;\\
&\to (\max\{0, m-1\}, \max\{1, k-1\}) \text{ with rate } \mu \min\{k, m\}\,,\ \text{if } m = R_{k-1} + 1\,.
\end{aligned}$$

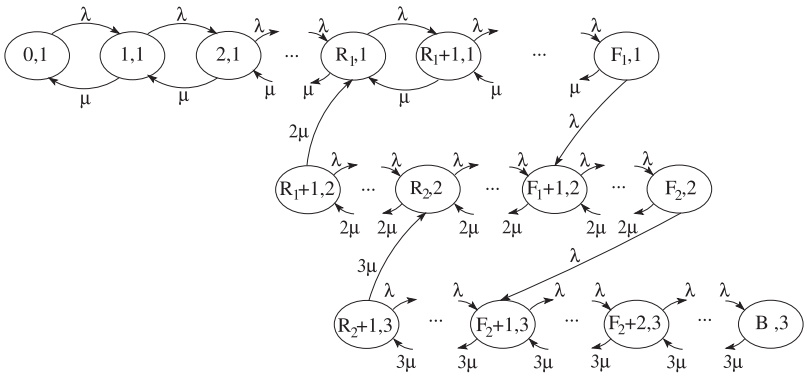


Fig. 3. Markov chain for $K = 3$.

In Figure 3, we give an example of these transitions for a maximum number of requests in the system equal to B and a number of levels K equal to 3.

4.1.1 Costs. Since the global cost $\bar{C}$ defined in Equation (1) is an expected cost that depends only on transitions and states at transitions times, it can be expressed using the stationary distribution of the Markov chain. For fixed $[F, R]$ we denote the cost $\bar{C}$ by $\overline{C}^{\pi}_{[F,R]}$ to illustrate the dependency on thresholds and stationary distribution. We have the following:

$$\overline{C}^{\pi}_{[F,R]} = \sum_{k=1}^{K} \sum_{m=R_{k-1}+1}^{F_k} \pi(m,k) \cdot \overline{C}(m,k)\,, \tag{4}$$

where $\pi(m,k)$ represents the stationary probability of state (m,k). For a given state (m,k) of the environment, Equation (2) can be rewritten and cloud provider pays a charged cost equal to

$$\overline{C}(m,k) = C_H \cdot m + C_S \cdot k + C_A \cdot \lambda \cdot \mathbb{1}_{\{m=F_k,\ k<K\}} + C_D \cdot \mu \cdot \min\{m,k\} \cdot \mathbb{1}_{\{m=R_{k-1}+1,\ 2\le k\le K\}} + C_R \cdot \lambda \cdot \mathbb{1}_{\{m=B,\ k=K\}}\,. \tag{5}$$

4.2 Thresholds Calculation Using Stationary Distributions of Markov Chain

We detail here the first framework to solve the stochastic optimization problem. This framework is a policy-based method in the sense that we explore the set of possible thresholds and for each of them we compute the objective function. The use of Markov chain to express the expected cost transform the stochastic problem into a deterministic one. We can thus apply heuristics for deterministic problem. We focus here on the expression of the optimization problem and on the detail of different algorithms that compute the optimal threshold policy minimising the expected global cost. We identify here the results presented in Reference [40] that we need to recall, on the one hand, to keep a self-content article and, on the other hand, since we improve them.

The deterministic optimisation problem. For a set of fixed parameters λ, μ, B, and K, and costs C_a, C_d, C_h, C_s, and C_r, the algorithms seeks to find the vector of thresholds $[F, R]$ such that the objective function is minimal. The objective function is the expected cost $\overline{C}^{\pi}_{[F,R]}$ defined by Equation (4). For any solution $[F, R]$, the computation of the expected cost depends on the stationary distribution π of the Markov chain $\{X(t)\}_{t\ge 0}$ induced by $[F, R]$. The thresholds must verify some given constraints resulting from the principle of hysteresis, that is, $R_k < F_k$, $R_{k-1} < R_k$, and $F_{k-1} < F_k$. Also, it is assumed that servers do not work for free, that is, $k < F_k$. Therefore, we have to solve

the constrained optimisation problem given by

$$\begin{array}{ll} \underset{F,R}{\text{Minimise}} & \overline{C}^{\pi}_{[F,R]} \\ \text{under constraints} & R_i < F_i \quad i = 1, \ldots, K-1. \\ & F_1 < F_2 < \cdots < F_{K-1} < B \cdot \\ & 0 \leq R_1 < R_2 < \cdots < R_{K-1} \\ & F_i \ , \ R_i \in \mathbb{N}. \end{array} \tag{6}$$

Since the cost function is non-convex [40] and threshold values are integer, we face up a combinatorial optimisation problem whose resolution time increases exponentially with the number of thresholds.

4.2.1 Local Search Heuristics. To solve this optimisation problem, several local search heuristics and one meta-heuristic have been developed in Reference [40]. Some of the heuristics are inspired by Kranenburg et al. [23] to solve a lost sales inventory problem. But, unlike hysteresis, there is only a single threshold by level, and Reference [40] presents the way to manage double thresholds. Furthermore, Reference [40] compares both the accuracy and the running time of numerous heuristics by numerical experiments. We keep here the two heuristics, **Best Per Level (BPL)** and **Neighborhood Local Search (NLS)**, that have the best results. We later present some improvement by combining them with a suitable initialization.

Best Per Level [40]. This algorithm first initialises $[F, R]$ with the lowest feasible value, i.e., $F_1 = 1, F_2 = 2, \ldots, F_{K-1} = K$ and $R_1 = 0, R_2 = 1, \ldots, R_{K-1} = K-1$. Then it improves each threshold in the following order: It starts with the first activation threshold F_1 by testing all its possible values taking into account the hysteresis constraints. A new value of F_1 that improves the global cost will replace the old one. Then it will move on to $F_2, F_3, \ldots, F_{K-1}, R_1, R_2, \ldots, R_{K-1}$. Once a loop is finished, it will restart again until the mean global cost is no longer improved.

Neighborhood Local Search [40]. This algorithm is the classical local search algorithm. We randomly initialise the solution $[F, R]$. Then we generate the neighborhood $\mathcal{V}(x)$ of a current solution x. Each neighboring solution is the same as the current solution except for a shift ± 1 for one of the thresholds. The algorithm explores all the neighborhood and returns the best solution among $\mathcal{V}$. Again, it loops the same process until the mean global cost is no longer improved.

4.2.2 Heuristics Using Queueing Model Approximations. We present a new heuristic that calculates a near-optimal solution very quickly. It can be considered a method in its own right, but we prefer to use it for the initialisation step in local search heuristics. This heuristic is called the *MMK approximation*, since it uses $M/M/k/B$ results to compute the costs of these close models.

Principle of the heuristic. The main idea of the heuristic is that, at each level k, we compute an approximation of the mean global cost by using the stationary distribution of the Markov chain of a $M/M/k/B$ queue model instead of using the Markov chain of Section 4.1. We thus have an approximation of the stationary distribution. In $M/M/k/B$, the stationary distribution of the Markov chain is given by a well-known closed formula. Thus, the computation of the distribution is done in a constant time due to the closed formula while the exact computation is much longer. Hence, we obtain an approximate solution of the expected cost very quickly. To find the best threshold m for which it is better to activate or deactivate, we proceed by comparing approximated costs of having k VMs activated with the one of having $k+1$ (respectively $k-1$) VMs activated added by the activation (respectively, deactivation) cost. Obviously the approximation of the stationary distribution can be very bad. However, we choose not to evaluate the accuracy of the average cost

approximation obtained, since several other parameters can alter the quality of the optimization. The whole method will be assessed later.

Design of the heuristic. The formulas of the stationary distributions in $M/M/k/B$ queues are usual in the field of queuing theory, we recall it now. Let $\hat{\pi}(k,m)$ be the stationary distribution of state (m,k) it is equal to

$$\hat{\pi}(k,m) = \hat{\pi}(k,0) \cdot \frac{\rho^m}{m!} \quad \text{for } 1 \le m \le k \quad \text{and} \quad \hat{\pi}(k,m) = \hat{\pi}(k,0) \cdot \frac{a^m \cdot k^k}{k!} \quad \text{for } k < m \le B,$$

where $\rho = \dfrac{\lambda}{\mu}$, $a = \dfrac{\rho}{k} < 1$ and where $\hat{\pi}(k,0)$ is

$$\hat{\pi}(k,0) = \left(\sum_{m=0}^{k-1} \frac{\rho^m}{m!} + \frac{k^k}{k!}(B-k+1) \right)^{-1} \text{ if } a = 1 \text{ and } \hat{\pi}(k,0) = \left(\sum_{m=0}^{k-1} \frac{\rho^m}{m!} + \frac{\rho^k}{k!} \frac{1-a^{B-k+1}}{1-a} \right)^{-1} \text{ if } a \neq 1.$$

Let us define $\hat{C}_k(m)$ as the approximate cost of having k VMs turned on and m requests in the system. It is computed similarly as Equation (5) but uses the approximation of the distribution function $\hat{\pi}(k,m)$. It is equal to

$$\hat{C}_k(m) = \hat{\pi}(k,m)\Big(m \cdot C_H + k \cdot C_S\Big) + \hat{\pi}(k,B) \cdot \lambda \cdot C_R\,;$$

we similarly define $\hat{C}^A_{k+1}(m)$ as the approximate cost of having m requests in level $k+1$ knowing that we have just activated a new VM by

$$\hat{C}^A_{k+1}(m) = \hat{\pi}(k+1,m)(m \cdot C_H + (k+1) \cdot C_S) + \hat{\pi}(k+1,B) \cdot \lambda \cdot C_R + \hat{\pi}(k,m-1) \cdot \lambda \cdot C_A\,.$$

Let us define $\hat{C}^D_k(m)$ as the approximate cost of having m requests in level k knowing that we have just deactivated a new VM by

$$\hat{C}^D_k(m) = \hat{\pi}(k,m)\,(m \cdot C_H + k \cdot C_S) + \hat{\pi}(k,B) \cdot \lambda \cdot C_R + \hat{\pi}(k+1,m+1) \cdot (k+1) \cdot \mu \cdot C_D\,.$$

We want to compare these costs to know whether we should activate, deactivate, or let the servers remain unchanged.

For the activation thresholds, we define $\phi^A_k(m) = \hat{C}_k(m) - \hat{C}^A_{k+1}(m)$. If $\phi^A_k(m) < 0$, then it is better to not activate a new server, while if $\phi^A_k(m) \ge 0$, then it is better to activate a new one. There is no evidence of the monotonicity of $\phi^A_k(m)$ in m; nevertheless, we choose the threshold by $F_k = \min\left\{m : \phi^A_k(m) \ge 0\right\}$. To compute the whole thresholds, all k are studied in an ascending order. For a fixed level k, we need to compute $\phi^A_k(m)$ for all m and stop when the function $\phi^A_k(m)$ is larger than 0.

For deactivation thresholds, we define $\phi^D_k(m) = \hat{C}_{k+1}(m) - \hat{C}^D_k(m)$. If $\phi^D_k(m) < 0$, then it is better to stay with $k+1$ servers, while if $\phi^D_k(m) \ge 0$, then it is better to deactivate a virtual machine. Similarly, there is no evidence about the monotonicity of $\phi^D_k(m)$, but we define $R_k = \min\left\{m : \phi^D_k(m) \ge 0\right\}$. The computation of the whole deactivation thresholds is similar to the activation ones.

4.2.3 Fluid Method. In Reference [31] for the $M/M/1$ queue and in Reference [12] for the $G/G/1$ queue a fluid approximation based on queueing results is used to determine activation and deactivation thresholds. The principle of the algorithm is similar to the MMK heuristic above, but the costs are approximated by the fluid approximation. We will use this algorithm alone to compare our methods with literature techniques, and we consider this method as the baseline. We define by *Fluid* the fluid approximation algorithm.

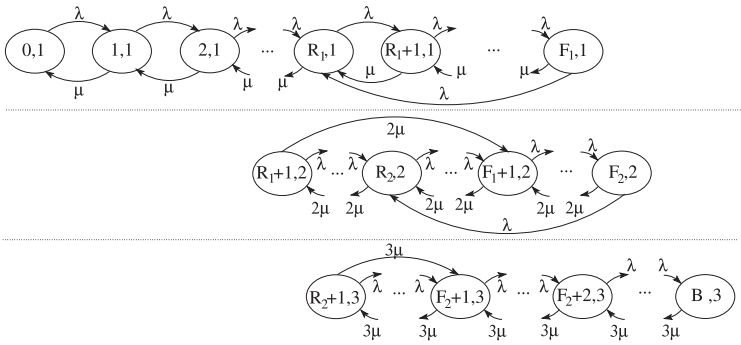


Fig. 4. Micro-chains aggregation.

4.2.4 Coupling Local Search Heuristics with Queuing Model Approximations. The initialisation of the local search heuristics presented in Section 4.2.1 either is totally random or simply chooses the bounds (lowest values or highest values) of the solution space. The aim here is to improve the speed and the accuracy of these heuristics by coupling them with the *MMKB approximation* used as an initialisation step. This initialisation aims at finding an initial solution that will be in the basin of attraction of the optimal solution. Starting with this solution improves the ability of local search algorithms to reach the best solution faster. We will show later that this new method behaves better in practice than the initialisation based on *Fluid approximation* adapted from Reference [31] in Reference [40]. We define by *Alg MMK* the coupled heuristic with MMKB approximation.

4.3 An Exact Aggregation Methods to Compute the Global Cost

Solving the Markov chain quickly becomes complex when the number of levels K increases. So we propose to apply the **Stochastic Complement Analysis (SCA)** method [29], which allows a simplified computation of the exact stationary probability. This approach is based on a decomposition and an aggregation of the Markov chain. The aim is to separate the K levels of the considered Markov chain into K independent sub-chains, called *micro-chains*, and then to solve each of them independently. We then build the aggregated Markov chain that has K states such that each state represents an operating level. This chain is called *macro-chain*.

4.3.1 Details about micro-chains and macro-chain. For any level k, such that $1 \leq k \leq K$, the micro-chain is built from the initial Markov chain by keeping the transitions inside the same level. We call *leaving state* for a level k the state where the number of requests m corresponds to a threshold that leads to an other level and *entering state* the state from which other levels can transit into the current level. As for the transitions between states of different levels, they are deleted and replaced by new transitions inside the *micro-chains* from a leaving state to an entering state of the initial chain. Let us illustrate this in Figure 3 for initial chain and in Figure 4 for the micro-chains. Inside level 2, the transitions with rate λ and those with rate 2μ are kept, while the transitions between level 2 and level 1 and those between level 2 and level 3 are removed. The deleted transitions between levels 2 and 3 are replaced by a transition inside level 2 between $(F_2, 2)$ and $(R_2, 2)$ with rate λ, and inside level 3 by a transition between $(R_2 + 1, 3)$ and $(F_2 + 1, 3)$ with rate 3μ. Similarly for transitions between levels 2 and 1, then the process is generalised for any levels.

The *macro-chain*, is represented by a K states Markov chain, such that each state is a level k (with $1 \leq k \leq K$). The *macro-chain* is a birth-death process for which computing the stationary probability is well known. Let Π be the stationary distribution of the *macro-chain* and π_k be the

stationary distribution of the *micro-chain* of level k (with $1 \leq k \leq K$). We denote by π the stationary distribution of the initial Markov chain. From the SCA method, the probability distribution π is computed as follows:

$$\forall\ (m,k) \in \mathbb{S},\ \pi(m,k) \ = \ \Pi(k) \cdot \pi_k(m)\,. \tag{7}$$

Numerical solving. The use of closed formulas suffers from numerical instability (see Reference [20]), so the expected distribution of micro- and macro-chains are computed with numerical methods (the *power method* implemented in the *marmoteCore* software [18]). Each micro-chain is solved separately, next the macro-chain, and, last, with Equation (7), the initial Markov chain. This aggregation method is an exact method. However, we want to validate numerically this aggregation method; indeed, the numerical errors in all the micro-chain computations could lead to bad numerical results. We compare the invariant distributions obtained with the aggregation process and those obtained by direct computations on the whole chain. The infinite norm of the difference of the distributions is always smaller than the precision of the software.

Computation of Aggregated Costs. The framework described above, allows to compute the global cost $\overline{C}^{\pi}_{[F,R]}$, defined by Equation (4), in an aggregated form based on the costs per level computed with *micro-chains*.

Theorem 1 (Aggregated Cost Function [40]). *Let $[F,R]$ be fixed thresholds, then the expected cost $\overline{C}^{\pi}_{[F,R]}$ of the system can be decomposed as*

$$\overline{C}^{\pi}_{[F,R]} \ = \ \sum_{k=1}^{K} \Pi(k) \cdot \overline{C}(k)\,, \qquad \textit{with}\ \ \overline{C}(k) = \sum_{m=R_{k-1}+1}^{F_k} \pi_k(m) \cdot \overline{C}(m,k)\,,$$

where $\overline{C}(k)$ represents the expected cost of level k.

Proof. We use Equation (7) in Equation (4) to get $\overline{C}^{\pi}_{[F,R]} = \sum_{k=1}^{K} \sum_{m=R_{k-1}+1}^{F_k} \pi_k(m) \cdot \Pi(k) \cdot \overline{C}(m,k)$. Because $\Pi(k)$ does not depend on m, it is removed from the second sum. This yields the average aggregated costs per level weighted by the *macro-chain* stationary distribution,

$$\overline{C}^{\pi}_{[F,R]} \ = \ \sum_{k=1}^{K} \Pi(k) \ \cdot \sum_{m=R_{k-1}+1}^{F_k} \pi_k(m) \cdot \overline{C}(m,k) \ = \ \sum_{k=1}^{K} \Pi(k) \cdot \overline{C}(k)\,. \qquad \square$$

Theorem 1 suggests an efficient approach for the global cost computation. Indeed, instead of having computations on a multidimensional Markov chain, we have computations over several one-dimensional Markov chains. We take advantage of this aggregated expression to improve the computation of the expected cost when only a single threshold changes.

Corollary 2 ([40]). *Let $[F,R]$ be a fixed vector of thresholds. It is assumed that the* micro-chains *and the costs per level associated with $[F,R]$ are already computed. The modification of a threshold F_k or R_k in $[F,R]$ only impacts the levels k and $k+1$ in the Markov chain. Thus, the computation of the new average cost only requires a new computation of π_k, π_{k+1}, and Π as well as $\overline{C}(k)$ and $\overline{C}(k+1)$.*

The intuition about the proof can be understood from the transition graph in Figure 3. When modifying F_2, it only impacts the distributions π_2 and π_3. We thus have to compute new $\overline{C}(2)$, $\overline{C}(3)$ and Π, while the last expected cost $\overline{C}(1)$ remains unchanged.

Proof. If we modify a threshold F_k at a given level k, then we only modify the entering and the leaving transitions of the micro-chain of level k and that of level $k+1$. All the other micro-chains are left unchanged, and their stationary distribution is also left unchanged; indeed, the dependence

between the levels is expressed in the macro chain. Then micro-chain of level k and $k+1$ should be computed as well as the macro chain. The use of Theorem 1 finishes the proof for the costs. Note that a more detailed proof can be found in Reference [40]. □

4.3.2 Aggregation for Local Search Algorithms. Local search algorithms are based on an exploration step in a set of feasible solutions that must be evaluated. This evaluation consists of calculating $C^{\pi}_{([F,R])}$. Although aggregation speeds up the stationary distribution computation, causing the neighboring solutions to differ from the current solution only by a slight change of one of their thresholds, to compute the stationary distribution of the neighbor we have to recalculate all the elements (micro- and macro-chain). This represents a large amount of time that depends on the number of neighboring solutions tested at each iteration and that increases when we consider large-scale scenarios (with large K and B).

To avoid so many computations for each neighboring solution, we exploit Corollary 2. As soon as we notice that a neighbor of a solution only differs by a simple change of a single threshold, then at each iteration, we only have to compute the stationary distributions and the corresponding average costs per level of the two micro-chains impacted by a change of a threshold. We no longer need to compute the distributions of the other micro-chains, since we can keep their average costs per level. The macro-chain still need to be solved, however. Its use will widely accelerate the algorithm without altering the accuracy.

Some algorithmic tricks must be implemented in order to ensure a proper and efficient running. Hence, the two micro-chains of the current solution that are modified when we study a neighborhood have to be stored, since they will be reused many times. By this way, the execution time of aggregated local search heuristics is reduced. This will be all the more effective when K increases.

5 COMPUTING POLICIES WITH MARKOV DECISION PROCESS

We consider here the multi-server queue of Section 3.1. We solve it with a MDP. We present the resolution method for computing the optimal policy that solves the problem defined by Equation (1) and the resolution method to find the optimal hysteresis policy for the same problem. A special concern is given to the correctness of such solution, since this receive very few treatment in the literature. In this section, we first describe the MDP and then the algorithms for solving.

5.1 The Semi Markov Decision Process Model

The dynamic control problem (1) is a continuous time process and thus should be solved using a **Semi Markov Decision Process (SMDP)**.

5.1.1 Elements of the SMDP. The state space is the one defined in Section 3: $\mathcal{S} = \{0, 1, \ldots, B\} \times \{1, \ldots, K\}$. Hence, a state $x \in \mathcal{S}$ is such that $x = (m, k)$, where m is the number of customers in the system and k the number of active servers. Similarly, the action space $\mathcal{A} = \{-1, 0, 1\}$ represents respectively: deactivate one machine, leave unchanged the active servers, or activate one machine.

The system evolves in continuous time, and at some time a transition occurs. When a transition occurs, the controller observes the current state and reacts to adapt the resources by activating or deactivating the virtual machines. The activation or deactivation are instantaneous, and then the system evolves until another transition occurs. Two events can occur: an arrival with rate λ that increases the number of customers present in the system by one or a departure that decreases the number of customers by one.

Let $x = (m, k)$ be the state and a the action; we define $N(k + a)$ as the real number of active VM after the action a was triggered. We have $N(k + a) = \min\{\max\{1, k + a\}, K\}$. It follows that the

transition rate is equal to

$$\begin{cases} \lambda & \text{if } y = \Big(\min\{m+1, B\}, N(k+a)\Big) \\ \mu \cdot \min\{m, N(k+a)\} & \text{if } y = \Big(\max\{0, m-1\}, N(k+a)\Big) \end{cases}.$$

5.1.2 Uniformised Transition Probabilities. We apply here the standard method to deal with continuous time MDP: the uniformisation framework. We follow the approach of Chapter 11 of Reference [34] to define the uniformised MDP. Roughly speaking, uniformisation consists in choosing a common transition rate for all transitions and all states. Thus, all transition times have the same distribution. To reach this common rate, we add pseudo events that are jumps of a state to itself. This modifies the transition probabilities of the initial problem.

We define $\Lambda(m, k, a)$ as the rate per state. We have $\Lambda(m, k, a) = \lambda + \mu \cdot \min\{m, N(k+a)\}$. From now on, any component denoted by "$\sim$" refers to the uniformised process. We thus define the uniformisation rate by $\tilde{\Lambda}$ with $\tilde{\Lambda} = \max_{(m,k,a)} \Lambda(m, k, a) = \lambda + K\mu$, which is the maximum transition rate. The transition probability from state x to state y when action a is triggered is denoted by $\tilde{p}(y|x, a)$ in the uniformised model.

For $x = (m, k)$, we have the following:

$$\tilde{p}(y|x,a) = \begin{cases} \dfrac{\lambda}{\tilde{\Lambda}} & \text{if } y = (m+1, N(k+a)) \text{ , arrival} \\ \dfrac{\mu \min\{m, N(k+a)\}}{\tilde{\Lambda}} & \text{if } y = (m-1, N(k+a)) \text{ , departure} \\ \dfrac{\tilde{\Lambda} - \Lambda(m,k,a)}{\tilde{\Lambda}} & \text{when } y = (m,k) \text{ , pseudo event} \\ 0 & \text{otherwise ,} \end{cases}$$

when $x = (m, k)$ such that $0 < m < B$ and a is arbitrary; when $x = (B, k)$ with $a \neq 0$, or when $x = (B, k)$ with $k \neq K$ and $a \neq +1$ or also when $x = (B, k)$ with $k \neq 1$ and $a \neq -1$; when $x = (1, k)$ with $a \neq 0$, or when $x = (1, k)$ with $k \neq K$ and $a \neq +1$, or also when $x = (1, k)$ with $k \neq 1$ and $a \neq -1$. We have the following:

$$\tilde{\Lambda} \times \tilde{p}(y|x,a) = \begin{cases} \mu \min\{m, N(k+a)\} & \text{if } y = (B-1, N(k+a)) \text{ , departure} \\ \tilde{\Lambda} - \mu \min\{m, N(k+a)\} & \text{when } y = (B,k) \text{ , pseudo event} \\ 0 & \text{otherwise ,} \end{cases}$$

when $x = (B, k)$ with $a = 0$, or when $x = (B, k)$ with $k \neq K$ and $a \neq +1$, or also when $x = (B, k)$ with $k \neq 1$ and $a \neq -1$. We have the following:

$$\tilde{\Lambda} \times \tilde{p}(y|x,a) = \begin{cases} \lambda & \text{if } y = (1, N(k+a)) \text{ , arrival} \\ \tilde{\Lambda} - \lambda & \text{when } y = (0,k) \text{ , pseudo event} \\ 0 & \text{otherwise ,} \end{cases}$$

when $x = (0, k)$ with $a = 0$, or when $x = (0, k)$ with $k \neq K$ and $a \neq +1$, or when $x = (0, k)$ with $k \neq 1$ and $a \neq -1$.

5.1.3 The Uniformised Stage Costs. We take the definition of costs of Section 3.1. Instantaneous costs are charged only once and are related to activation, deactivation, and losses. Accumulated costs are accumulated over time and are related to consumption and holding cost. After uniformisation (Chapter 11.5 of Reference [34]), the expectation of the transition is included in Equation (2)

of the charged cost. We obtain for $x = (m, k)$:

$$\tilde{c}(x,a) = \left(C_A \mathbb{1}_{\{a=1\}} + C_D \mathbb{1}_{\{a=-1\}}\right) \frac{\Lambda(x,a)}{\tilde{\Lambda}} + \frac{\lambda}{\tilde{\Lambda}} C_R \mathbb{1}_{\{m=B\}} + \frac{1}{\tilde{\Lambda}} \left(N(k+a) \cdot C_S + m \cdot C_H\right).$$

Objective function. By replacing the expression of the charged cost $\tilde{c}(x, a)$, defined above, in Equation (1), the objective function translates for all $x \in \mathcal{S}$ into the following:

$$C^q(x) = \lim_{N \to \infty} E^q \left[\frac{1}{N} \sum_{t=0}^{N-1} \tilde{c}(x_t, \pi(x_t)) \mid x_0 = x \right],$$

for a given policy q. The minimum of C^q is equal to the minimum of $\bar{C}$.

5.2 Solving the MDP

5.2.1 Classification of the SMDP. Our SMDP is an average cost model. This is why the expected stage costs depend on the recurrent properties of the underlying Markov chain that is generated by a deterministic policy. A classification is then necessary to study them. We use the classification scheme of Reference [34]. For communicating multichain SMDP, Reference[19] insures that uniformisation breaks neither structural properties of the generator nor the optimality of the policy computed in the uniformised model.

Definition 5.1 (Chapter 8.3.1 in Reference [34]). An MDP is as follows:

(1) *Unichain* if the transition matrix corresponding to every deterministic stationary policy is unichain, that is, it consists of a single recurrent class, plus a possibly empty set of transient states;
(2) *Multichain* if the transition matrix corresponding to at least one deterministic stationary policy contains two or more recurrent classes;
(3) *Communicating* if, for every pair of state x and y in $\mathcal{S}$, there exists a deterministic stationary policy q under which y is accessible from x, that is, $p_q^n(y, x) > 0$ for some $n \geq 1$;

Proposition 2. *The MDP is multichain. There is a stationary deterministic policy with monotone hysteresis properties that induces a corresponding Markov chain with more than two different recurrent classes.*

Proof. We assume that $K \geq 2$ and let k be such that $k \in [1, \ldots, K]$. We build a policy such that it activates a new server only if the number of servers is smaller than $k - 1$; in level k, we have neither activation nor deactivation; it deactivates a server only if the number of servers is larger than $k + 1$. This defines the policy q as follows. For any level l such that $l < k - 1$, we have only activation. This means that there exists $m \in [0, \ldots, B]$ such that $q(m', l) = 1$ for any $m \leq m'$ and $q(m, l) \leq q(m, l - 1)$. For level k, we have $q(m, k) = 0$ for all $m \in [0, \ldots, B]$. For any level l with $l > k + 1$, we have only deactivation. There exists $m \in [0, \ldots, B]$ such that $q(m', l) = -1$ for any $m' \leq m$ and $q(m, l) \geq q(m, l + 1)$.

Hence, starting from any state such that $l \leq k - 1$, the highest attainable level is $k - 1$. The set of states for which $l = k - 1$ is a recurrent class. Equally, starting from any state of the level k, let us stay in this level.

At last, starting from any state such that $l \geq k + 1$, then the lowest attainable level is $k + 1$. Therefore, we have three recurrent classes: the level $k - 1$, the level k, and the level $k + 1$ (two recurrent classes for $K = 2$). □

Lemma 3. *The MDP is communicating. There exists a stationary isotone hysteresis policy such that the corresponding Markov chain is irreducible.*

Proof. We exhibit such a policy. Let q be the deterministic stationary policy such that the thresholds l_k are defined by $l_k = 0$ and the thresholds L_k by $L_k = B$ for all k. In this policy, as soon as we are in a state (B, k) we can move to $(B, k+1)$, and as soon as we are in a state $(0, k)$ we can move to $(0, k-1)$. In a given level k, all the states are reachable from any state and, by the way, the states (B, k) and $(0, k)$. Thus, the induced Markov chain is irreducible, since any state at any level can can be reached from another one. Therefore, we have that $p_q^n(y, x) > 0$ for some $n \geq 1$ for all couples (x, y). Therefore, the MDP is communicating. □

Bellman equations. From Reference [34], in the *multichain* case, the Bellman equations are composed of two equations. If we plug uniformized transitions and costs into the two Bellman equations, then we have in the uniformised model the two following optimality equations:

$$\min_{a \in A} \left\{ \sum_{y \in \mathcal{X}} \tilde{p}(y \mid x, a) C(y) - C(x) \right\} = 0 \text{ and } U(x) = \min_{a \in B_x} \left\{ \tilde{c}(x, a) - \frac{C(x)}{\tilde{\Lambda}} + \sum_{y \in \mathcal{X}} \tilde{p}(y \mid x, a) \cdot U(y) \right\}$$

for all $x \in \mathcal{X}$, where

$$B_x = \left\{ a \in \mathcal{A} \;\middle|\; \sum_{y \in \mathcal{X}} \tilde{p}(y \mid x, a) C(y) = C(x) \right\}.$$

It could be noticed that in the unichain case, $B_x = \mathcal{A}$ and that the two equations reduce to only the second one. These two non-linear equation systems should be numerically solved to find $U(x)$ and $\rho(x)$. Once these terms are approximated we deduce the optimal policy with

$$q(x) = \arg\min_{a \in B_x} \left\{ \tilde{c}(x, a) + \sum_{y \in \mathcal{X}} \tilde{p}(y \mid x, a) \cdot U(y) \right\}.$$

5.2.2 Algorithms. We describe here the choice of the algorithms used to solve this multichain and communicating model. Computing multichain model is a more complicated task than in the unichain case. We first show that we actually can use some unichain algorithms due to the communicating properties of our SMDP, and then we present two structured algorithms based on hysteresis properties.

Unichain algorithms. From Reference [34], there exists a *multichain policy iteration algorithm* that solves multichain models. It requires solving two equation systems and thus is time-consuming (complexity is doubled compared to unichain models). However, by Proposition 3, our MDP is communicating and in Theorem 8.3.2 of Reference [34] it is proved that unichain value iteration algorithm converges to the optimal value in communicating models. This property allows us to use the unichain value iteration algorithm.

There is also, in Reference [34], a policy iteration algorithm for communicating models. It requires to start with an initial unichain policy and its inner loop roughly differs from the unichain case to keep some properties. We decide to use here algorithms based on unichain policy iteration. There does not exist theoretical guarantee of their convergence, but we showed in numerical experiments that they always converge to the same policy than value iteration.

Four different usual unichain algorithms will be considered: *Value Iteration*, *Relative value iteration*, *Policy Iteration modified*, and *Policy Iteration modified adapted*, which adapts its precision in the policy evaluation step. They are respectively referred to as *VI*, *RVI*, **Policy Iteration (PI)**, and *PI Adapt*. The algorithms are all described in Reference [34] and are already implemented in the software [18].

Structured Policies algorithm. We now integrate hysteresis properties in the algorithms. Two classes of policies have been investigated: Double Level class (Definition 3.1) and Monotone Hysteresis class (Definition 3.2). Policy Iteration algorithm has two major steps: the first one is a greedy improvement of the policy, and the second one the evaluation of the policy. We modify the policy improvement step of policy iteration by plugging hysteresis assumptions instead of the greedy improvement. This allows us to test fewer actions at each iteration and to speed up the algorithm. Two algorithms are implemented: one for the double level properties (referred as **Double Level Policy Iteration (DL-PI)**) and one for the hysteresis properties (referred as **Hysteresis Policy Iteration (Hy-PI)**).

However, we do not have the theoretical guarantee that these methods converge, first, because we do not theoretically know if hysteresis policies are optimal for our model, and, second, because the underlying PI also has no convergence guarantees in multichain. Nevertheless, in the numerical experiments we made (see Section 8.2.2) all the optimal policies returned by classical algorithms have hysteresis properties. Furthermore, all MDP algorithms considered here (structured as well as classical) returned the same solution. This therefore underlines the interest of considering such hysteresis policies especially since the gain in running time is obvious as observed in the experiments.

Computation of the hysteresis thresholds. The non-structured MDPs (also called simple MDPs) do not assume any restrictions for their policy research. Thus, they return the optimal policy and so return a decision rule q^*, which gives the optimal action to take but not the thresholds. Therefore, we need to test the hysteresis property and to compute the l and L hysteresis thresholds consistently with Definition 3.2. Let q^* be the optimal policy returned by the PI algorithm. We check if q^* is monotone; if not, then the optimal policy is not hysteresis. If it is monotone, then we proceed as follows. We consider, for all k, the set $\{m \mid q^*(m,k) = 1 \text{ and } q^*(m-1,k) = 0\}$. If the set is empty, then $L_{k+1} = \infty$, else if it is of size 1 and then $k+1 = m$; otherwise, when the size is larger than 2 there are multiple values for a threshold, and the policy is not hysteresis. Also, we consider, for all k, the set $\{m \mid q^*(m+1,k) = 0 \text{ and } q^*(m,k) = -1\}$. If the set is empty, then $l_{k+1} = 0$; if the size is 1, then $l_{k+1} = m$ and otherwise the policy is not hysteresis.

6 THEORETICAL COMPARISON BETWEEN THE TWO APPROACHES MC AND MDP

We studied here the theoretical differences between the two models, the MC approach and the MDP approach. Although they sound very similar, these two models present some rather subtle theoretical differences with some consequences. Numerical comparisons will be carried out in Section 8. We compare the dynamical behaviour of the two models and their optimality. Let us recall (for clarity) that $(F_1, \ldots, F_{K-1})$ and $(R_1, \ldots, R_{K-1})$ are the thresholds of the MC model and that $(L_2, \ldots, L_K)$ and $(l_2, \ldots, l_K)$ are the SMDP thresholds.

6.1 Different Temporal Sequence of Transitions

We study here the difference in temporal behavior between transitions. This difference is slight and has no effect on average costs; nevertheless, it induces a difference on the returned values of the thresholds. In the two systems, there are two kinds of transitions: *natural transitions*, which are due to events (departures or arrivals) and *triggered transitions*, which are caused by the operator (activation or deactivation). These two transitions are instantaneous. Due to their intrinsic definitions, the models considered here do not observe the system at the same epochs. Hence, in the Markov chain approach, the system is observed just before a natural transition while in the MDP

the system is observed just after a natural transition. More formally, let us assume that x is the state before a natural transition: It is seen by the Markov chain. Then the transition occurs and the state changes instantaneously in x', that is, the state seen by the MDP. The controller reacts and the triggered transition occurs; thus, the system moves in x'', in which the system remains until the next event that will cause the next natural transition. Since state changes are instantaneous, this has no impact on costs.

When thresholds are fully comparable, we have the following lemma.

Lemma 4. *When $\forall\, k$, we have that $L_k \neq \infty$ and $l_k \neq 0$, and then hysteresis thresholds of Markov chain and SMDP hysteresis can be inferred from each other. Let $(F_1, \ldots, F_{K-1})$ and $(R_1, \ldots, R_{K-1})$ be the thresholds of the MC model, and let $(L_2, \ldots, L_K)$ and $(l_2, \ldots, l_K)$ be the SMDP thresholds. Then, for all $k \in \{1, \ldots, K-1\}$, we have $L_{k+1} - 1 = F_k$ and $l_{k+1} + 1 = R_k$.*

Proof. We make the proof for an activation threshold. Let us assume that the state just before a transition is $x = (F_k, k)$; if an arrival event occurs, then the system moves instantaneously in a state $x' = (F_k + 1, k)$. According Definition 3.4, a virtual machine is activated, and then the system instantaneously moves again in $x'' = (F_k + 1, k + 1)$. Observe now that the state x' (according Definition 3.2) is the state in which the SMDP observes the system and takes its activation decision. Thus, we have $L_{k+1} = F_k + 1$. The proof works similarly for deactivation. □

6.2 Number of Activated Resources

We will see now that the hysteresis model formulated in Definition 3.4 from Lui [29] used in the deterministic approach has worse results and that it does not activate the same number of VM as policies obtained by MDP algorithms. Recall that a strictly isotone assumption (see Definition 3.3) is required in MC approaches to keep the size of the chain constant [17].

Proposition 3 (Suboptimality of MC Approach). *The minimum computed with the MC approach is greater than or equal to the minimum computed with structured SMDP (structured by hysteresis). In turn, the minimum computed with structured SMDP is greater than or equal to the one computed with simple MDP.*

Proof. The MC approach (Section 4), deals with *strictly isotone* policies (*i.e.,* $0 = l_1 < l_2 < \cdots < l_K \leq B$ and $0 = L_1 < L_2 < \cdots < L_K \leq B$). However, the SMDP model with hysteresis considers isotone policy (see Definition 3.3). Last, simple SMDP does not assume any structural property at all. Since the constraints are more relaxed, then the set of strictly isotone policy in the MC model is smaller than or equal to that of SMDP with hysteresis isotone policy assumption that is smaller than or equal to that of single SMDP. □

In Markov chain models, it is not possible to activate (respectively, deactivate) only a subset of the VM. Indeed, because L_K cannot be infinite, all the K servers should be activated. However, since inequalities are strict, then $l_K \geq K$, and the only state in which there is only one active server is restricted to the level $k = 1$. During the numerical experiments in Section 8, we identify numerous cases in which strictly isotone policies are not optimal. This is mainly due to the phenomenon of non-activation or non-deactivation, i.e., it appears in some cloud scenarios that the optimal policy does not require to pass in all system levels. Hence examples of non-optimality are found in two specific categories of the whole categories described below. We noticed that these categories depend on the relative values of the parameters (λ, μ) between them. However, we do not know how to precisely quantify their borders but can define different classes to partition the different cloud systems.

Definition 6.1. These categories are as follows:

(1) *Medium arrival case*: All *VMs* are turned on and turned off in both approaches. This occurs when the system load is *medium*, i.e., the arrival rate λ is close to the service rate $k \times \mu$.
(2) *Low arrival case*: All *VMs* are turned on and turned off in MC, while in MDP some *VMs* will never be activated. This occurs when the system load is *low*, i.e., the arrival rate λ is very small compared to the service rate $k \times \mu$.
(3) *High arrival case*: All *VMs* are turned on and turned off in MC, while in MDP some *VMs* will never be deactivated. This occurs when the system load is *high*, i.e., the arrival rate λ is very large compared to the service rate $k \times \mu$.

7 REAL MODEL FOR A CLOUD PROVIDER

Now we provide real environment values to our models. To our knowledge, in the literature, n unified model including energy, quality of service, and real traffic parameters does not exist. We had to merge several models to build one. Henceforth, we propose to build a global cost taking into account both real energy consumption, real financial costs of virtual machines, and SLA simultaneously from separate works. We think that the model presented is sufficiently generic to represent a large class of problems and is enough to give a relevant meaning to all parameters. With our algorithms and this model, the cloud owner can generate meaningful optimised costs in real cases. Parameter values used in experiments can be found in Section 8.3.

7.1 Cost-aware Model with Energy Consumption Traces and Real Prices

Measurements of energy consumption. Many measurements are performed on the real data center *grid5000* in Reference [7], and energy consumption data are obtained for VMs hosted on physical servers in Reference [25]. We keep these values in watts for energy consumption of virtual machines.

Financial Cost. To keep the relevance of our *cost-aware* model, we must represent a financial cost. Henceforth, we transform energy consumption given in watts into a financial cost based on the price in euros of the kilowatt-hours in France fixed by a national company. To obtain the financial values of the other prices, we observe the commercial offers of providers [2]. This gives us the operational costs.

Service Level Agreement. We propose to modify the performance part of our cost function for capturing the realistic scheme of the Service Level Agreement. Actually, we have to give a concrete meaning to our holding cost, since it does not translate directly from usual SLA models. Indeed, in SLA contracts, it could be stated that when the response time exceeds a pre-defined threshold T_{SLA}, then penalty costs must be paid. Here the penalty C_p is the price that a customer pays for 1 hour of service of a virtual machine (full reimbursement) [2], and the threshold corresponds to a maximal time to process a request.

With our model, it is possible to define a holding cost that models the penalty to pay when the QoS is not satisfied. We focus on the mean response time in the system where the response time is denoted by R_T. The SLA condition translates in $E[R_T] \leq T_{SLA}$. We introduce the mean number of customers $E(N)$, and with Little's law we get $E[N] \leq T_{SLA} \cdot \lambda$. We thus may define a customer threshold N_{SLA} such that $N_{SLA} = T_{SLA} \cdot \lambda$. This customer threshold is the maximum number of requests that the system can accept to keep the required *QoS* satisfied. Therefore, each time $m > N_{SLA}$, the cloud provider will pay a holding cost penalty of C_p per customer. We also assume for simplicity that the penalty cost is equal to the rejection cost ($C_p = C_r$), but it can take a different value.

Global cost formula. We express similarly as Equation (5) the expected global cost charged to the cloud provider:

$$\overline{C}(m,k) = C_p \cdot (m - N_{SLA}, 0)^+ + C_S \cdot k + C_p \cdot \lambda \cdot \mathbb{1}_{\{m=B,k=K\}} + C_A \cdot \lambda \cdot \mathbb{1}_{\{m=F_k,\ k<K\}} + C_D \cdot \mu \cdot \min\{m,k\} \cdot \mathbb{1}_{\{m=R_{k-1}+1,\ 2\leq k\leq K\}} + C_{static}.$$

In the energy consumption part, C_{static} represents the static financial cost derived from the idle energy consumption of the physical server that hosts the virtual machines. The remaining energy part of the cost function is left unchanged with the activation, deactivation, and energy costs of a VM defined above.

7.2 Real Packet Traffic and CPU Utilisation

For obtaining concrete values of parameters λ and μ, we must search workload traces with real scenarios in public cloud. Here the traces come from MetaCentrum Czech National Grid data [11].

8 NUMERICAL EXPERIMENTS

This part is devoted to numerical experiments and the comparison among all previous algorithms. We have implemented all methods in C++, with the stand-alone library [18].

8.1 Experiments Design and Results

All the experiments were carried out on a platform built on an Intel Xeon X7460 processor, with 6×4 cores at 2.66 GHz and 16 GB of RAM. Numerical experiments have been performed for different cloud model parameters. System parameters were taken such that they cover a large number of different situations to get a complete comparison. We defined four scenarios with different scales: Scenario A: $K = 3$, $B = 20$; Scenario B: $K = 5$, $B = 40$; Scenario C: $K = 8$, $B = 60$; and Scenario D: $K = 16$, $B = 100$. We call *Instance* a set of parameters $\lambda, \mu, C_A, C_D, C_H, C_S, C_R$. The algorithms have been launched on 46,656 instances. Costs $(C_A, C_D, C_H, C_S, C_R)$ were taking values in $[0.5, 1, 2, 5, 10, 20]$, C_p in $[1, 10, 100, 1000, 5000, 10000]$, and queueing parameters λ, μ in $[0.5, 1, 2, 5, 10, 20]$. We rank the different algorithms by assessing their accuracy and their running time. We first studied numerical experiments in the MC approach by comparing the different heuristics by these two criteria. For Scenario A, accuracy is computed based on optimal solution generated by exhaustive search. For higher-scale scenarios (B,C,D), we compute accuracy regarding the best solution among the heuristics. Then we compared all the (MDP) algorithms with the value iteration solution (convergence is known from Section 5.2.2) before comparison with the heuristics to compare both approaches. We finally ran the experiments for a concrete cloud scenario with real data, showing how we can effectively benefit from the best methods to calculate optimal thresholds.

8.2 Experiments Analysis

We display in Table 2 the comparisons for MC heuristics and MDP algorithms. For each algorithm, we display for each scenario the running time in seconds in the first column and the accuracy in the second column.

8.2.1 Assessing Heuristics Approaches. We want to assess the selected heuristics of Reference [40] and the gain of the improvements proposed.

Aggregation strongly improves algorithms time speed. For the same efficiency, we first observe a significant time saving provided by the aggregation technique. As expected, the improvement in Scenario A is small, but the gain increases as the size of the system rises. Henceforth, the execution

Table 2. Numerical Experiments for Comparison of Heuristics and MDP Algorithm

Models	Algorithms	Scenario A		Scenario B		Scenario C		Scenario D	
		Time (sec)	% Opt	Time (sec)	% Min	Time (sec)	% Min	Time (sec)	% Min
MC Heuristics	**BPL**	0,063 sec	96,9 %	2,684 sec	86,81%	37,23 sec	79,57 %	757 sec	55,39%
	BPL Agg	0,047 sec	96,9 %	1,163 sec	86,81%	10,27 sec	79,57 %	186 sec	55,39%
	BPL-MMK Agg	0,036 sec	97,78 %	0,53 sec	96,59 %	5,16 sec	95,39 %	31,7 sec	87,24 %
	NLS	0.043 sec	92,70 %	1,007 sec	62,24 %	12,38 sec	43,51 %	257 sec	20,44 %
	NLS Agg	0,034 sec	92,70 %	0,458 sec	62,24 %	4,05 sec	43,51 %	61 sec	20,44 %
	NLS-MMK Agg	0,014 sec	96,32 %	0,06 sec	93,58 %	0,9 sec	89,54 %	8,39 sec	76,17 %
	Fluid	0.00049 sec	49.11 %	0.0031 sec	42.32 %	0.011 sec	37.14 %	0.099 sec	26.09 %
		Time (sec)	% Opt	Time (sec)	% Opt	Time (sec)	% Opt	Time (sec)	% Opt
MDP	**VI**	0.0057 sec	100 %	0.021 sec	100 %	0.0406 sec	100 %	0,0944 sec	100 %
	RVI	0.0057 sec	100 %	0.021 sec	100 %	0.0406 sec	100 %	0,095 sec	100 %
	PI	0.0028 sec	100 %	0.0124 sec	100 %	0.0214 sec	100 %	0.0606 sec	100 %
	PI Adapted	0,0023 sec	100 %	0,0117 sec	100 %	0,0201 sec	100 %	0,0583 sec	100 %
	DL-PI	0,00115 sec	100 %	0,0072 sec	100 %	0,0105 sec	100 %	0,0452 sec	100 %
	Hy-PI	0,00113 sec	100 %	0,0069 sec	100 %	0,0100 sec	100 %	0,0442 sec	100 %
		Time (sec)	% Opt	Time (sec)	% Opt	Time (sec)	% Opt	Time (sec)	% Opt
Comparison	**Fluid**	0.00049 sec	36.85 %	0.0031 sec	34.47 %	0.011 sec	33.27 %	0.099 sec	20.81 %
	NLS-MMK Agg	0.014 sec	64.82 %	0.06 sec	61.52 %	0,9 sec	59,82 %	8,39 sec	52,61 %
	BPL-MMK Agg	0,036 sec	65,15 %	0,53 sec	62.65 %	5,16 sec	61,69 %	31,7 sec	57,1 %
	DL-PI	0.00115 sec	100 %	0.0072 sec	100 %	0.0105 sec	100 %	0.0452 sec	100 %
	Hy-PI	0.00113 sec	100 %	0.0069 sec	100 %	0.0100 sec	100 %	0.0442 sec	100 %

Algorithms in blue refer to the baselines of the literature.

time is divided by about 1.3 when $K = 3$ and $B = 20$ and by about 3.8 when $K = 16$ and $B = 100$ (e.g., BPL algorithm passes from 757 seconds to 186 seconds).

Impact of the M/M/k/B approximation. Coupled with heuristics *BPL* and *NLS*, M/M/k/B approximation brings significant improvement considering the efficiency and the time execution. For a large-scale case (Scenario D in Table 2), the *BPL MMK Agg* provides the best solution for 87.24% of the instances in 31.7 seconds, which is the best heuristic in terms of time–accuracy ratio. We can notice, in all cases, that coupled heuristics always have better accuracy than the heuristic alone. Moreover, the initialisation technique does not only improve efficiency but also time execution. For example, in Scenario C, *BPL Agg* has a mean time of 10,27 seconds for 79,57 % accuracy while *BPL-MMk Agg* obtains 95,39% accuracy in 5.16 seconds, providing a major gain both in time and in efficiency.

8.2.2 Comparison between MDP Algorithms. From Section 5.2.2, we theoretically know that value iteration algorithm converges to the exact solution. We observe in these numerical experiments that all MDP algorithms obtain 100% accuracy (optimal solution being given by value iteration) in all scenarios. Thus, all MDP algorithms converge to the optimal solution. Concerning the running time, we observe that policy iteration algorithms *PI* and *PI Adapted* are twice as good as value iteration on all studied scenarios. Furthermore, we see that the execution time of the structured MDP algorithms *DL-PI* and *Hy-PI* is halved compared to *PI Adapted*. Recall that VI is often the standard resolution method for MDP. We show that the use of hysteresis policies can speed up the resolution by a factor between 2 and 4 This leaves us to conclude that integration of policy with structural properties strongly accelerates the convergence.

8.2.3 Comparison between MDP and MC Approaches. We now compare which approach works best to obtain optimal thresholds quickly. We first observe that the accuracy difference is striking; indeed, the MC approach is optimal only about half the time. From Section 6.2, we knew that depending on the queueing parameters, the MDP approach could not activate or deactivate certain virtual resources, whereas the MC approach activate all the levels that let MC optimal costs go higher due to extra costs generated by additional activation or deactivation thresholds. We wanted

Table 3. High-scale Data-center Resolution with MDP Algorithm Hy-PI

	$K = 3$, $B = 20$	$K = 5$, $B = 40$	$K = 8$, $B = 60$	$K = 16$, $B = 100$	$K = 32$, $B = 200$	$K = 64$, $B = 400$	$K = 128$, $B = 800$	$K = 256$, $B = 1{,}600$	$K = 512$, $B = 3,200$	$K = 1{,}024$, $B = 6{,}400$
State Space	60	200	480	1,600	6,400	25,600	102,400	409,600	$1,6 \times 10^6$	$6,5 \times 10^6$
Hy-PI	0.0002	0.0019	0.0274	0.0517	0.3023	2,4238	37,97	329.54	2,327	22,850
BPL-MMK Agg	0.036	0.53	5.16	31.7	590	ND	ND	ND	ND	ND

to assess the ratio of cases where the MC approach is not optimal. Also, we point out that MDP algorithms are faster than heuristics in the four scenarios. Hence, for small scenarios *Hy-PI* is about 100 times faster than the best heuristic and about 200 times faster for large scenarios. Therefore, the MDP approach is significantly better than the MC approach.

Comparison with baseline algorithms. We compare our methods with existing techniques from the fluid method presented in Section 4.2.3. *Fluid Agg* (fluid method coupled with Markov chain aggregation) is strongly faster than the other MC methods (almots 100 times faster), but we observe that this technique has very poor performances considering the accuracy compared to other heuristics (in scenario D 20% of the solution returned are optimal comparing to 76% of *NLS-MMK AGG*). Comparatively with SMDP, the fluid approach is then 2 times faster than hysteresis MDP for small instances. It has the same performance for a state space size of 240 and has worse performance for larger case (MDP is 2 times faster for a state space size of $1,600$). But MDP is always optimal while *Fluid Agg* has a poor accuracy.

Therefore, we propose here a method that behaves better than the baseline algorithms for the accuracy and that becomes faster for large instances.

8.2.4 High-scale Simulations MDP. We ran numerical simulations for high-scale scenarios (real size data center) with best MDP algorithm *Hy-PI* to assess its performance. Simulations were performed on pre-selected costs and queuing parameters. We consider sizes K and B that are larger than these in some previous works, e.g., Reference [31]. Results are displayed in Table 3. We also display the state space size, and we fill the table with *ND* (Not Defined) when the execution time is above 10 hours. We show that the best local search heuristic (*BPL-MMK Agg*) already suffers in the case of 64 VM and 400 customers. We notice that it does not scale.

Observe that for small data centers (64 VM and 400 customers), the optimal policy can be computed with SMDP in a reasonable time in practice (2 seconds). However, for large data centers, some further research should be done, since the time represents around 6 hours. Hence, for high size of the state space the proposed algorithm fails to give a result in an reasonable time. Further research is needed. But the perspectives we have in mind: aggregation for MDP or approximate dynamic programming are beyond the scope of this article.

We note that Reference [45] uses MDP to solve a problem of deactivation of non-mandatory micro services. There is not running time given in the article (as usual) but the size of the state space can be estimated around 80000 states. In our work, such a size gives a running time smaller than 30 seconds.

8.3 Numerical Experiments for Concrete Scenarios

Data and scenarios. We define now numerical values to the real model of Section 7. Several case-studies as well as several performance and metrics curves are used to select the values. Throughout this section, the time unit is the hour. The platform comes from Reference [25]: The physical hardware is a Taurus Dell PowerEdge R720 server having a processor Intel Xeon e5-2630 with 6×2 cores up to 2.3GHz. The processor hosts virtual machines with a hypervisor that is a software that enables the sharing of a cloud provider's physical computing and memory resources across

Table 4. Parameter Values for Three Scenarios

Parameter	Model A	Model B	Model C
$\#VMs$	3	6	12
Instances	a1.xlarge	t3.small	t2.micro
$\#vCPUs$	4	2	1
RAM(Go)	8	2	1
C_p	0.0914€/h	0.0211€/h	0.0118€/h
C_S	0.00632€/h	0.00316€/h	0.00158€/h
$C_A = C_D$	0.00158€	0.00079€	0.00032€
C_{Static}	0.0158€/h	0.0158€/h	0.0158€/h
B	100	100	100
λ	50 req/h	50 req/h	50 req/h
μ	20 req/h	10 req/h	5 req/h

multiple virtual machines. One core is considered to be one vCPU, and cores are distributed equally among the virtual machines: Twelve VMs with one core per VM, 6 VMs with two cores per VM, and 3 VMs with four cores per VM. These three different cases are selected such that we can use the prices of Amazon EC2 instances [2].

Energy consumption values come from Reference [25]. Physical hardware consumption is around 100 W. The dynamic consumption of the virtual machines is stated in accordance with the number of cores per VM. Namely, the watt consumption for activation, deactivation, and use is different, as it depends on the characteristics of the virtual machines hosted on the physical server. Hence, 1 VM with 4 vCPU that executes a request requires 40W to work while its activation and deactivation take a power of 10 W. Last, all the energy consumption values are transformed in financial costs.

Some workload samples that come from real traces of the MetaCentrum Czech National Grid are detailed in Reference [11]. We select some samples to build concrete daily scenarios for huge size requests. Hence, we pick a number of arrivals per hour of $\lambda = 50$ and a number of served requests per hour as $\mu = 5, 10$, or 20. Table 4 details all the parameter values for each model.

We provide the mean runtimes of the best algorithm (Hy-PI) among concrete experiments, 0.00127 seconds for Model A, 0.0073 seconds for Model B, and 0.0372 seconds for Model C. The optimal solution is computed in a short time and therefore can allow us to recompute new policy when the demand is evolving to dynamically adapt to the need.

Mean global costs given SLA parameter or arrival rate. From numerical simulations, it could be noticed first that, for a fixed arrival rate λ, when the SLA is not so strict, then the cost decreases. Second that, when the response time set by the SLA rises, then the operating costs for the cloud providers are reduced. Indeed, providers will pay fewer penalties to clients and fewer power bills for running VMs. Moreover, we observe that Model C is always better than Model B and Model A for any possible values of N_{SLA} and for any values of λ given a fixed SLA parameter. We can conclude that it is always better to decompose physical machine in several virtual resources, since it allows a more efficient resource allocation in dynamic scenario. Indeed, the more virtual resources you have, the less you will pay per utilisation, since they require less CPU consumption. Moreover, it allows a more flexible system where you can easily adapt virtual resources to the demand.

Thresholds given SLA parameter or arrival rate. We also observe that Model C always activates a second virtual resource before the two other models B and A, for any fixed values of λ and

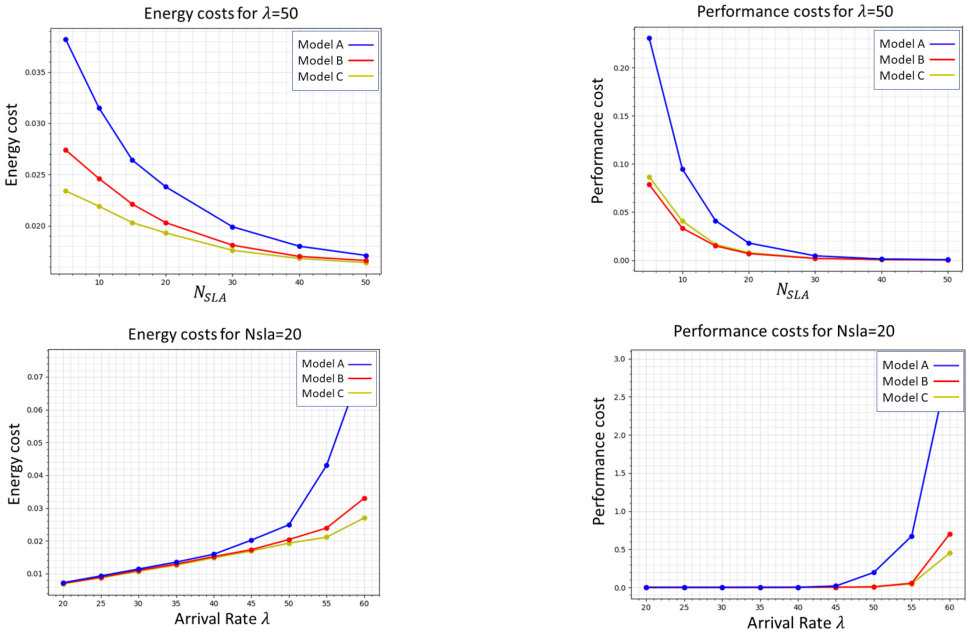


Fig. 5. Energy and performance costs given fixed SLA or arrival rate.

SLA parameter N_{SLA}. Furthermore, when the arrival rate λ is fixed, then all models will activate later when the SLA is less restrictive. Last, when the SLA parameter is fixed, we notice that the first activation threshold decreases when λ increases. Indeed, if the demand is growing, then it requires more resources and faster activation of virtual resources.

Cost evaluation. Figure 5 presents an example ($K = 8, B = 60$) that exhibits the evolution of the financial cost per hour according to a given N_{SLA} or to the load. We can see in the figure the impacts of such parameters on performance and energy costs.

9 CONCLUSIONS AND FURTHER WORK

In this article, we have compared theoretically and numerically two different modelling approaches to minimise the global cost integrating both performance and energy consumption in a dynamic auto-scaling cloud system. We exhibit that hysteresis presents a significant improvement and that the best deterministic heuristics are strongly outperformed by SMDP models (almost 700 times slower). The strong result of the article is to propose effective methods (i.e., precise and fast) for calculating thresholds. The relevance of this study for a cloud provider is to provide an auto-scaling policy very quickly to minimise its financial cost (around 38 seconds for 128 VMs and 800 customers). Our next steps will be to enlarge the model by considering a system with several queues in tandem (representing service function chaining) and including more general arrival processes as burst traffic predicted from a measurement tool. We will also investigate hierarchical MDP techniques to tackle very large-scale cloud systems. We plan also to build a cloud prototype to evaluate the performances of our policies in a real environment.

REFERENCES

[1] Amazon. 2018. AWS Auto Scaling. Retrieved from http://aws.amazon.com/autoscaling/.
[2] Amazon. 2019. Amazon EC2 Pricing. Retrieved from http://aws.amazon.com/ec2/pricing/.

[3] D. Ardagna, G. Casale, M. Ciavotta, J. F. Pérez, and W. Wang. 2014. Quality-of-service in cloud computing: Modeling techniques and their applications. *J. Inf. Secur. Appl.* 5, 11 (2014), 17.
[4] J. R. Artalejo, A. Economou, and M. J. Lopez-Herrero. 2005. Analysis of a multiserver queue with setup times. *Que. Syst.* 51, 1-2 (2005), 53–76.
[5] N. M. Asghari, M. Mandjes, and A. Walid. 2014. Energy-efficient scheduling in multi-core servers. *Comput. Netw.* 59, 11 (2014), 33–43.
[6] N. Baer, A. Al Hanbali, J.-K. van Ommeren, and R. J. Boucherie. 2021. A successive censoring algorithm for a system of connected QBD-processes. *Ann. Operat. Res.* 310 (2021), 389–410.
[7] A. Benoit, L. Lefèvre, A.-C. Orgerie, and I. Rais. 2018. Reducing the energy consumption of large scale computing systems through combined shutdown policies with multiple constraints. *Int. J. High Perform. Comput. Appl.* 32, 1 (2018), 176–188.
[8] Guilherme Domingues, Gabriel Mendonça, Edmundo De Souza E Silva, Rosa M. M. Leão, Daniel S. Menasché, Ori Rottenstreich, Mostafa Dehghan, and Don Towsley. 2021. The role of hysteresis in caching systems. *ACM Trans. Model. Perform. Eval. Comput. Syst.* 6, 1, Article 2 (May 2021), 38 pages. https://doi.org/10.1145/3450564
[9] A. Gandi, M. Harchol-Balter, and I. Adan. 2010. Server farms with setup costs. *Perf. Eval.* 67, 11 (2010), 1123–1138.
[10] Hamoun Ghanbari, Bradley Simmons, Marin Litoiu, Cornel Barna, and Gabriel Iszlai. 2012. Optimal autoscaling in a IaaS cloud. In *Proceedings of the 9th International Conference on Autonomic Computing (ICAC'12)*. Association for Computing Machinery, New York, NY, 173–178. https://doi.org/10.1145/2371536.2371567
[11] D. Guyon, A-C. Orgerie, C. Morin, and D. Agarwal. 2019. Involving users in energy conservation: A case study in scientific clouds. *Int. J. Grid Util. Comput.* 10, 3 (2019), 272–282.
[12] R. Han, M.M. Ghanem, L. Guo, Y. Guo, and M. Osmond. 2014. Enabling cost-aware and adaptive elasticity of multi-tier cloud applications. *Fut. Gener. Comput. Syst.* 32, C (2014), 82–98.
[13] Z. Han, H. Tan, G. Chen, R. Wang, Y. Chen, and F. Lau. 2016. Dynamic virtual machine management via approximate Markov decision process. In *Proceedings of the IEEE International Conference on Computer Communications (INFOCOM'16)*. 1–9.
[14] Peter G. Harrison, Naresh M. Patel, and William J. Knottenbelt. 2016. Energy–performance trade-offs via the EP queue. *ACM Trans. Model. Perform. Eval. Comput. Syst.* 1, 2 (Jun 2016), 31.
[15] S. K. Hipp and U. D. Holzbaur. 1988. Decision processes with monotone hysteretic policies. *Operat. Res.* 36, 4 (1988), 585–588.
[16] Shay Horovitz and Yair Arian. 2018. Efficient cloud auto-scaling with SLA objective using Q-learning. In *Proceedings of the 6th International Conference on Future Internet of Things and Cloud*. IEEE, 85–92. https://doi.org/10.1109/FiCloud.2018.00020
[17] O. C. Ibe and J. Keilson. 1995. Multi-server threshold queues with hysteresis. *Perf. Eval.* 21, 3 (1995), 185–213.
[18] A. Jean-Marie. 2017. marmoteCore: A Markov modeling platform. In *Proceedings of the 11th EAI International Conference on Performance Evaluation Methodologies and Tools (VALUETOOLS'17)*. ACM, New York, NY, 60–65.
[19] L. Jianyong and Z. Xiaobo. 2004. On average reward semi-Markov decision processes with a general multichain structure. *Math. Operat. Res.* 29, 2 (2004), 339–352.
[20] M. M. Kandi, F. Aït-Salaht, H. Castel-Taleb, and E. Hyon. 2017. Analysis of performance and energy consumption in the cloud. In *European Workshop on Performance Engineering*. Springer, 199–213.
[21] A. A. Khan, M. Zakarya, and R. Khan. 2019. Energy-aware dynamic resource management in elastic cloud datacenters. *Simul. Model. Pract. Theory* 92 (2019), 82–99.
[22] M. Y. Kitaev and R. F. Serfozo. 1999. M/M/1 queues with switching costs and hysteretic optimal control. *Operat. Res.* 47, 2 (1999), 310–312.
[23] A. A. Kranenburg and G. J. van Houtum. 2007. Cost optimization in the (S-1, S) lost sales inventory model with multiple demand classes. *Oper. Res. Lett.* 35, 4 (2007), 493–502.
[24] J. Krzywda, A. Ali-Eldin, T. E. Carlson, P.-O. Ostberg, and E. Elmroth. 2018. Power-performance tradeoffs in data center servers: DVFS, CPU pinning, horizontal and vertical scaling. *Fut. Gener. Comput. Syst.* 81 (2018), 114–128.
[25] M. Kurpicz, A.-C. Orgerie, and A. Sobe. 2016. How much does a VM cost? Energy-proportional Accounting in VM-based Environments. In *Proceedings of the Euromicro Conference on Parallel, Distributed and Network-Based Processing (PDP'16)*. IEEE, 651–658.
[26] N. Lee and V. G. Kulkarni. 2014. Optimal arrival rate and service rate control of multi-server queues. *Que. Syst. Theory Appl.* 76, 1 (2014), 37–50.
[27] Wen-Hwa Liao, Ssu-Chi Kuai, and Yu-Ren Leau. 2015. Auto scaling strategy for Amazon web services in cloud computing. In *Proceedings of the International Conference on Smart City/SocialCom/SustainCom*. IEEE, 1059–1064.
[28] T. Lorido-Botran, J. Miguel-Alonso, and J. A. Lozano. 2014. A review of auto-scaling techniques for elastic applications in cloud environments. *J. Grid Comput.* 12 (2014), 559–592.

[29] J. C. S. Lui and L. Golubchik. 1999. Stochastic complement analysis of multi-server threshold queues with hysteresis. *Perf. Eval.* 35 (1999), 19–48. Issue 1.
[30] T. Mastelic and I. Brandic. 2015. Recent trends in energy-efficient cloud computing. *IEEE Cloud Comput.* 2, 1 (2015), 40–47.
[31] I. Mitrani. 2013. Managing performance and power consumption in a server farm. *Ann. Operat. Res.* 202 (2013), 121–134.
[32] Observatoire de l'industrie Electrique 2017. *Le cloud, les data centers et l'énergie*. Observatoire de l'industrie Electrique.
[33] S. Parminder, G. Pooja, J. Kiran, and N. Anand. 2019. Research on auto-scaling of web applications in cloud: Survey, trends and future directions. *Scal. Comput.: Pract. Exp.* 20, 2 (2019), 32.
[34] M. L. Puterman. 1994. *Markov Decision Processes: Discrete Stochastic Dynamic Programming*. Wiley, New York.
[35] S. Shorgin, A. Pechinkin, K. Samouylov, Y. Gaidamaka, I. Gudkova, and E. Sopin. 2015. Threshold-based queuing system for performance analysis of cloud computing system with dynamic scaling. In *Proceedings of the International Conference on Numerical Analysis and Applied Mathematics*, Vol. 1648. AIP, 3.
[36] D.-P. Song. 2013. *Optimal Control and Optimization of Stochastic Supply Chain Systems*. Springer-Verlag.
[37] D. S. Szarkowicz and T. W. Knowles. 1985. Optimal control of an M/M/S queueing system. *Operations Research* 33, 3 (1985), 644–660.
[38] Salman Taherizadeh and Vlado Stankovski. 2019. Dynamic multi-level auto-scaling rules for containerized applications. *Comput. J.* 62, 2 (2019), 24.
[39] J. Teghem. 1986. Control of the service process in a queueing system. *Eur. J. Operat. Res.* 23, 2 (1986), 141–158.
[40] T. Tournaire, H. Castel, E. Hyon, and T. Hoche. 2019. Generating optimal thresholds in a hysteresis queue: A cloud application. In *Proceedings of the International Symposium on Modeling, Analysis and Simulation of Computer and Telecommunication Systems (IEEE Mascots'19)*. IEEE, 283–294.
[41] T. Tournaire, H. Castel-Taleb, and E. Hyon. 2021. Optimal control policies for resource allocation in the cloud: Comparison between Markov decision process and heuristic approaches. arXiv:2104.14879. Retrieved from https://arxiv.org/abs/2104.14879.
[42] D. P. Warsing, W. Wangwatcharakul, and R.E. King. 2013. Computing optimal base-stock levels for an inventory system with imperfect supply. *Comput. Operat. Res.* 40, 11 (2013), 2786–2800.
[43] C.-H. Wu, W.-C. Lee, J.-C. Ke, and T-H. Liu. 2014. Optimization analysis of an unreliable multi-server queue with a controllable repair policy. *Comput. Operat. Res.* 49 (2014), 83–96.
[44] M. Xu and Rajkumar Buyya. 2017. Energy efficient scheduling of application components via Brownout and approximate Markov decision process. In *International Conference on Service-Oriented Computing*. Springer, 206–220.
[45] M. Xu, A.N. Toosi, B. Bahrani, R. Razzaghi, and M. Singh. 2019. Optimized renewable energy Use in green cloud data centers. In *International Conference on Service-Oriented Computing*. Springer, 314–330.
[46] Z. Yang, M.-H. Chen, Z. Niu, and D. Huang. 2011. An optimal hysteretic control policy for energy saving in cloud computing. In *Proceedings of the IEEE Global Communications Conference (GLOBECOM'11)*. 1–5.

Received 9 March 2022; revised 9 May 2023; accepted 23 May 2023